

FEELZLIKE · V0.3 TO V1.0

The Replit Prompt Playbook

Every prompt, sprint and affiliate setup needed to take FeelZlike from v0.3 to launch — copy-paste into Replit Agent.

6

Sprints

35+

Prompts

228

Curated entries

8

Affiliate programs

The Skier & Snowboarder's Bible

Prepared for James · Navigate Work · Built in Replit · feel-zlike-weather-app.replit.app

Contents

Part 1 How to use this playbook

- Replit Agent vs Assistant
- The golden workflow between prompts
- Order matters · Prerequisites

Part 2 Import the curated data (Prompt 0.1)

- 228 entries across 6 towns
- Loader script + JSON import

Part 3 · Sprint 1 Critical bug fixes

- 1.1 Fix the Cooma Coaches → Yudanaka leak
- 1.2 Explore API permanent loading state
- 1.3 EmptyStateCard component
- 1.4 Hourly forecast wiring

Part 4 · Sprint 2 Stay section build-out

- 2.1 OpenAPI schema for Stays
- 2.2 StayCard component
- 2.3 FilterBar + URL state
- 2.4 Map view (Leaflet)
- 2.5 AffiliateLinks helper
- 2.6 Stay page wiring + telemetry

Part 5 · Sprint 3 Eat section build-out

- 3.1 Eat OpenAPI schema
- 3.2 EatCard + cuisine badges
- 3.3 Eat FilterBar (cuisine, price, openNow)
- 3.4 openNow logic with timezone
- 3.5 Eat page + reservations links

Part 6 · Sprint 4 Differentiators

- 4.1 AI day-personalization
- 4.2 Powder Window predictor
- 4.3 Webcams gallery
- 4.4 Wind-hold prediction
- 4.5 Powder Factor score

Part 7 · Sprint 5 Alerts & subscriptions

- 5.1 Email subscriptions
- 5.2 Web push notifications
- 5.3 Cron triggers
- 5.4 Manage Alerts page

Part 8 · Sprint 6 Polish & launch

- 6.1 Homepage redesign
- 6.2 PWA install
- 6.3 SEO + sitemap

- 6.4 Sentry + analytics

Part 9 Affiliate program guides

- Trip.com · Agoda · Booking · Expedia
- Rakuten (via Involve Asia) · Tripadvisor · Jalan · Airbnb
- Prompt 9.1 — activate IDs once approved

Part 10 Appendices

- Schema reference (Stay/Eat)
- URL & route cheatsheet
- Workflow tips · Glossary

PARTS 1 & 2

How to use & data import

Set up your workflow with Replit Agent and load all 228 curated entries before any feature work.

Part 1 — How to use this playbook

What this playbook is

A sequenced set of **Replit Agent prompts** that, taken together, build out FeelZlike from its current v0.3 state to a launch-ready v1.0 product covering Snowy Mountains (NSW) and Yamanouchi (Nagano) regions with full Stay & Eat sections, hourly forecasts, AI day-planning, powder alert subscriptions, and live affiliate revenue.

Every prompt is copy-paste ready. You feed it to Replit Agent, review what it produces, test it locally, commit, then move to the next prompt.

How to use Replit Agent vs Replit Assistant

Replit Agent (the autonomous one — "Build me a..."):

- Use for prompts marked **[AGENT]** in this playbook
- Best for: building new features end-to-end across multiple files
- Give it the full prompt as written; let it run; review the diff before accepting

Replit Assistant (the inline pair-programmer — chat in the sidebar):

- Use for prompts marked **[ASSISTANT]** in this playbook
- Best for: targeted edits to a single file you have open
- Open the relevant file first, then paste the prompt into the Assistant chat

If you're not sure which to use, **default to Agent** — every prompt in this playbook is written to work with Agent.

The golden workflow (do this between every prompt)

- 1 **Commit your current state** before pasting the next prompt

```
git add -A && git commit -m "checkpoint before prompt X.Y"
```

BASH

- 2 **Paste the prompt** into Replit Agent
- 3 **Watch the diff** — don't auto-accept; read what Agent proposes to change
- 4 **Test locally** — every prompt has a "How to test" section. Do it.
- 5 **If broken** — use the "If it goes wrong" recovery prompt, OR `git reset --hard HEAD` and try again
- 6 **Commit the result** with the suggested commit message
- 7 **Move to the next prompt**

This rhythm keeps you safe. You can always roll back to the last green commit.

Order matters

The prompts are sequenced deliberately. **Don't skip ahead.** Sprint 2 (Stay) depends on Prompt 0.1 (data import). Sprint 3 (Eat) shares the filter components from Sprint 2. Skipping creates rework.

Suggested cadence:

- **Sprint 1** (4 prompts) — one focused day
- **Sprint 2** (6 prompts) — 2–3 days
- **Sprint 3** (5 prompts) — 2 days
- **Sprint 4** (5 prompts) — 2–3 days
- **Sprint 5** (4 prompts) — 2 days
- **Sprint 6** (4 prompts) — 2 days

Total: ~14 working days for a single developer working alongside Agent.

Before you start: prerequisites

- FeelZlike Replit project loaded (the monorepo)
- `pnpm install` runs cleanly
- `pnpm dev` serves the app locally
- You can see the existing v0.3 app at the live URL
- Git working tree is clean (`git status` is empty)
- The data package from this playbook (228 stays + eats) is ready to upload (see Part 2)

Part 2 — Import the curated data

Before any prompt runs, the 228 curated entries (107 stays + 121 eats across 6 towns) need to be in your repo. This is a one-time setup.

Step 1 — Upload the data package

The package was built and shared with you separately as `feelzlike_data_v0.4.zip` (~305 KB). Inside it has:

```
by_region/snowy_mountains/{jindabyne,berridale,cooma}/{stays.json, eats.json}
by_region/yamanouchi/{yudanaka,shibu_onsen,yomase}/{stays.json, eats.json}
feelzlike_data.json           ← single combined file
stays.csv / eats.csv         ← human-review CSVs
affiliate_links.js           ← reference deep-link helper (we'll port to TS)
README.md / SCHEMA.md
```

In Replit:

- 1 Right-click the project root in the file tree
- 2 **Upload file** → select `feelzlike_data_v0.4.zip`
- 3 Open the Shell and run:

BASH

```
mkdir -p artifacts/feelzlike/src/data/curated
unzip -o feelzlike_data_v0.4.zip -d artifacts/feelzlike/src/data/curated
rm feelzlike_data_v0.4.zip
```

You should now see `artifacts/feelzlike/src/data/curated/by_region/...` populated.

Step 2 — Run Prompt 0.1

This prompt wires the data into your codebase the right way: Zod schemas in `lib/api-zod` (so it's typed), loader hooks in the app, and a re-export so any page can pull stays/eats by town.

Prompt 0.1 — Wire the curated data into the typed data layer [AGENT]

GOAL

Take the 228-entry JSON data set we just uploaded at `artifacts/feelzlike/src/data/curated/` and make it available to the React app as fully-typed, queryable data following the existing monorepo conventions.

You are working in the FeelZlike monorepo. The structure is:

- `artifacts/feelzlike` — the Vite + React 19 web app (uses Wouter, TanStack Query, Tailwind v4, Radix, Zod, Leaflet)
- `lib/api-spec` — OpenAPI hand-edited spec
- `lib/api-zod` — generated Zod schemas (auto-generated from `api-spec`)
- `lib/api-client-react` — generated React Query hooks (auto-generated from `api-spec`)
- `lib/feelzlike-shell` — shared region chrome
- `lib/feelzlike-dashboard` — shared "Today's Call" dashboard

I just uploaded curated Stay + Eat data to `artifacts/feelzlike/src/data/curated/`. The structure is:

- `by_region/snowy_mountains/{jindabyne,berridale,cooma}/{stays.json, eats.json}`
- `by_region/yamanouchi/{yudanaka,shibu_onsen,yomase}/{stays.json, eats.json}`
- `feelzlike_data.json` — single combined file
- `SCHEMA.md` — full schema reference

Coverage: 107 stays + 121 eats, total 228 entries.

Please do the following, exactly:

1. Read `SCHEMA.md` and `feelzlike_data.json` to understand the schema (NSW vs Japan have different extra fields).
2. Create `artifacts/feelzlike/src/types/stayEat.ts` with Zod schemas for:
 - `StaySchema` (with discriminated union on `country`: 'AU' adds `drying_room/ski_storage/pet_friendly/distance_to_*`; 'JP' adds `onsen/tattoo_policy/meal_plan/yukata_provided/walk_min_to_yudanaka_station`)
 - `EatSchema` (with discriminated union: 'AU' adds `apres_ski/takeaway/groceries`; 'JP' adds `vegetarian_friendly/kid_friendly`)
 - `BookingLinksSchema`, `HoursSchema`, etc. as needed
 - Export TypeScript types: `type Stay = z.infer<typeof StaySchema>`, etc.
 - Export `TownSlug` union, `RegionSlug` union from the actual data
3. Create `artifacts/feelzlike/src/data/index.ts` that:
 - Imports all 6 `by_region/*/stays.json` and `eats.json` as static JSON (Vite supports this natively)

- Validates each through the Zod schemas at module load (throws on invalid data – fail fast)
 - Exports typed accessors: `getStaysByTown(town)`, `getEatsByTown(town)`, `getAllStays()`, `getAllEats()`, `getTowns(region)`, `getRegions()`
4. Create `artifacts/feelzlike/src/hooks/useStayEat.ts` with TanStack Query hooks:
- `useStays(town)` – wraps `getStaysByTown` with `useQuery`; `queryKey` `['stays', town]`
 - `useEats(town)` – wraps `getEatsByTown` with `useQuery`; `queryKey` `['eats', town]`
 - Even though data is static, using Query keeps the same calling convention as the rest of the app and makes future migration to API-backed data trivial
5. Add to `artifacts/feelzlike/src/data/__tests__/stayEat.test.ts` (if vitest is configured) a smoke test that asserts:
- Total stays count === 107
 - Total eats count === 121
 - All 6 expected towns are present
 - Every stay has a non-empty `booking_links` object
 - Schema validation passes for every entry
6. DO NOT modify `lib/api-spec`, `lib/api-zod`, or `lib/api-client-react` – the data is static for now, kept inside `artifacts/feelzlike`. We can migrate to API-backed later.
7. Run `pnpm typecheck` from the repo root and fix any type errors in the new files.
8. Print a final summary: files created, total entry count loaded, any schema warnings.

Files Replit will create:

- `artifacts/feelzlike/src/types/stayEat.ts`
- `artifacts/feelzlike/src/data/index.ts`
- `artifacts/feelzlike/src/hooks/useStayEat.ts`
- `artifacts/feelzlike/src/data/__tests__/stayEat.test.ts`

How to test:

```
pnpm --filter @workspace/feelzlike typecheck
pnpm --filter @workspace/feelzlike test 2>/dev/null # if tests are wired
pnpm --filter @workspace/feelzlike dev
```

BASH

Then in your browser console on the running app:

```
fetch('/').then(() => console.log('app loaded'))
```

JS

The app should still render the v0.3 UI exactly as before — no visible changes yet. We just made the data available to the codebase.

What to commit: `feat(data): wire 228-entry curated stay/eat dataset with Zod validation`

If it goes wrong:

- "Vite says it can't import JSON" → Add `"resolveJsonModule": true` to `artifacts/feelzlike/tsconfig.json` `compilerOptions`
- "Zod schema fails on real data" → The schema is too strict; relax the field that's failing to `.optional().nullable()` and re-run

- "Test count is off" → Check `feelzlike_data.json.meta.towns` for actual per-town counts; my count of 228 may be 1–2 off if curation files were updated

SPRINT 1

Critical bug fixes

Fix region-isolation leaks, Explore API failures, hourly forecast wiring, and add safety guards.

Part 3 — Sprint 1: Critical bug fixes (Week 1)

The audit surfaced four issues that undermine trust the moment a user encounters them. Fix these before any new feature lands.

Prompt 1.1 — Fix the Cooma Coaches ☐ Yudanaka Transport data leak [AGENT]

GOAL

Currently, navigating to `/yamanouchi/yudanaka/transport` shows "Cooma Coaches — Snowy Mountains Bus Service" with an Australian phone number. This is NSW data leaking into the Japan region. We need region-isolated transport data and a defensive layer that prevents this class of bug ever happening again.

In the FeelZlike monorepo, the Transport page at `/yamanouchi/yudanaka/transport` currently displays Australian bus operator data (Cooma Coaches, phone (02) 6452 1584) intended for the Snowy Mountains region. This is a region-isolation bug.

Please:

1. Locate the Transport page component (likely in `artifacts/feelzlike/src/pages` or `src/regions`). Find where transport providers are sourced.
2. Identify the root cause: most likely a hardcoded or improperly-filtered list that doesn't filter by region. Read the actual code, do not guess.
3. Refactor transport data into a structured per-region file:
 - Create `artifacts/feelzlike/src/data/transport/snowy_mountains.ts`
 - Create `artifacts/feelzlike/src/data/transport/yamanouchi.ts`
 - Each exports a typed `TransportProvider[]` array
 - Define the `TransportProvider` type in `artifacts/feelzlike/src/types/transport.ts`:
`{ id, name, name_local?, type: 'bus'|'train'|'shuttle'|'taxi'|'rental_car', operator, phone, website, route_summary, schedule_url?, regions: RegionSlug[] }`
4. Seed `snowy_mountains.ts` with: Cooma Coaches, Snowliner Coaches, Greyhound services to Cooma, Murrays Coaches Canberra-Cooma, NSW TrainLink Xplorer to Cooma, Snowy Mountains Airport shuttle services. Phone numbers and websites must be verified – use null where unknown rather than guessing.
5. Seed `yamanouchi.ts` with the actually-relevant providers: Nagano Dentetsu (Nagaden) Yudanaka line from Nagano Station, Nagano Snow Shuttle (snowmonkeyresorts.com runs this), Shiga Kogen Express Bus (Nagano Bus), Nagano Station Snow Monkey Express bus, Hakuba/Yamanouchi shuttle services where they exist. Include kanji names where available.
6. Build a region-isolation guard: in `artifacts/feelzlike/src/lib/regionGuard.ts` add a function `assertProvidersForRegion(providers, region)` that throws (in dev) and Sentry-warns (in prod) if any provider's `regions[]` doesn't include the requested region. Call this in the Transport page component before rendering.
7. Audit the Stay, Eat, Roads, and Explore pages for the same class of bug – anywhere data is rendered without an explicit region filter. Add the same `regionGuard` check. List anything you find in the final summary.

8. Test by visiting `/yamanouchi/yudanaka/transport` – should show ONLY Japanese transport providers. `/snowy-mountains/jindabyne/transport` should show ONLY Australian providers.

9. Add a Sentry breadcrumb when `regionGuard` catches a leak (so we get telemetry if this ever recurs).

Run `pnpm typecheck` and `pnpm dev`. Report what you found and changed.

How to test:

- 1 Visit `/yamanouchi/yudanaka/transport` → only Japanese providers, no Cooma Coaches
- 2 Visit `/snowy-mountains/jindabyne/transport` → only Australian providers
- 3 In Replit Shell: `grep -r "Cooma Coaches" artifacts/feelzlike/src/data/yamanouchi 2>/dev/null` → should return nothing

What to commit: `fix(transport): region-isolate transport providers + add regionGuard`

If it goes wrong:

- "regionGuard throws in production" → switch dev throw to `console.error` + Sentry capture; never throw in prod
- "Some pages now blank" → likely the audit (step 7) added guards that fire on existing leaky data; fix the data, not the guard

Prompt 1.2 — Fix the Explore API permanent loading state [AGENT]

GOAL

The Explore section (`/yamanouchi/yudanaka/explore`) shows skeleton placeholders forever. The API call is failing silently. Fix the failure, add proper error UI, and add Sentry coverage.

The Explore page at `/yamanouchi/yudanaka/explore` is permanently stuck showing skeleton card placeholders. The data fetch is failing without surfacing an error.

Please:

1. Find the Explore page component and the data hook it uses. Identify the API endpoint or data source.
2. Reproduce the failure – open browser DevTools → Network → reload the explore page. Capture the failing request and the actual error (4xx? 5xx? CORS? wrong URL?).
3. Diagnose:
 - If it's a missing/wrong endpoint in `api-spec` → add it to `lib/api-spec`, regenerate `api-zod` and `api-client-react` with the existing codegen scripts (find them in `scripts/` or in the workspace root `package.json`), then update the page to use the new hook
 - If it's a working endpoint that returns malformed data → fix the schema or the parser
 - If the data source isn't built yet → make this honest: render an "Explore is launching soon" empty-state component instead of an infinite skeleton
4. Add proper error UI: if the fetch fails, show a friendly error card with a retry button

(use the existing Radix-styled components – check shadcn config in artifacts/feelzlike/components.json for the patterns).

5. Wire Sentry: capture any fetch error with `Sentry.captureException` so we get production telemetry.
6. If the page is going to stay empty for now, hide the "Explore" link from the sidebar nav (in `lib/feelzlike-shell`) until data is wired up – better than a broken-feeling empty page.
7. Run `pnpm typecheck`.

Tell me exactly what was broken and how you fixed it.

How to test:

- 1 Visit `/yamanouchi/yudanaka/explore` → either real content, or a clean "launching soon" state, never a stuck skeleton
- 2 DevTools → Network → no failing requests
- 3 DevTools → Console → no uncaught errors

What to commit: `fix(explore): handle fetch failure + graceful empty state`

Prompt 1.3 — Graceful empty states for Stay & Eat (until Sprint 2/3 land) [AGENT]

GOAL

Stay and Eat currently render "No stays found near this town." That's worse than not having the page at all — it tells users we have nothing for them. Replace with branded "Launching this week" cards that build anticipation. (Once Sprints 2 & 3 land, these get replaced with real listings.)

The Stay and Eat pages currently show bare "No stays found near this town" / "Nothing found near this town yet" text whenever data is empty.

Please create a shared `EmptyStateCard` component at `artifacts/feelzlike/src/components/EmptyStateCard.tsx` that:

- Takes props: `{ icon: LucideIcon, title: string, body: string, eta?: string, ctaLabel?: string, ctaHref?: string }`
- Uses the existing Tailwind v4 + Radix design system
- Renders a centered card with icon, headline, body copy, optional ETA badge, optional CTA button
- Matches the FeelZlike aesthetic (deep navy text, sky-blue accents, lots of breathing room)

Then wire it into:

- Stay page: `<EmptyStateCard icon={Bed} title="Stays launching this week" body="We're curating hand-picked ryokan, lodges and apartments near {town}. In the meantime, browse availability on your favourite booking site." eta="ETA: Next 7 days" />`
- Show the existing 8-site search availability buttons BELOW the empty state card
- Eat page: `<EmptyStateCard icon={Utensils} title="Eats launching shortly" body="We're putting together a hand-picked guide to the best izakaya, ramen, cafes and bars in {town}. Watch this space." eta="ETA: Next 14 days" />`
- Optional CTA on Eat: a "Suggest a spot" `mailto` link to a feedback address (placeholder:

```
feedback@feelzlike.com)
```

The `EmptyStateCard` should be reusable across the app (we'll use it for Roads, Webcams, Lift Status etc. as we build those out).

Replace any other "Nothing found" / "Coming soon" plain text in the codebase with this component too.

Run `pnpm typecheck`.

How to test:

- 1 Visit `/yamanouchi/yudanaka/stay` → branded card, not bare text
- 2 Visit `/snowy-mountains/jindabyne/eat` → same treatment
- 3 Resize to mobile width → card stays readable

What to commit: `feat(ui): branded EmptyStateCard for unfinished sections`

Prompt 1.4 — Add hourly forecast to mountain pages [AGENT]

GOAL

Mountain pages currently show daily 6-day forecasts only. Skiers need hourly data to plan when to be on the chair (snow timing, wind picking up, visibility windows). Open-Meteo (which the app already uses) provides hourly data — wire it in.

Mountain pages currently show a 6-day forecast but no hourly breakdown. Open-Meteo's API supports hourly data via the `&hourly=` parameter for variables: `temperature_2m`, `apparent_temperature`, `snowfall`, `rain`, `windspeed_10m`, `windgusts_10m`, `cloudcover`, `visibility`, `freezinglevel_height`, `weathercode`.

Please:

1. Find the existing Open-Meteo fetch logic in `artifacts/feelzlike/src` or in `lib/feelzlike-dashboard`. Identify where mountain weather is fetched.
2. Extend the fetch to request hourly variables for the next 48 hours. Don't break the existing daily forecast – add to it.
3. Add a Zod schema for the hourly response in `artifacts/feelzlike/src/types/weather.ts`:

```
HourlyForecastSchema { time: string[], temperature_2m: number[], apparent_temperature: number[], snowfall: number[], rain: number[], windspeed_10m: number[], windgusts_10m: number[], cloudcover: number[], visibility: number[], freezinglevel_height: number[] }
```
4. Build an `HourlyForecast` component at `artifacts/feelzlike/src/components/HourlyForecast.tsx` that:
 - Renders the next 24-48 hours in a horizontal scrollable strip (use `Embla` – already a dep)
 - Each hour cell: time, weather icon, temp, snowfall (if any), wind speed
 - Highlight the "Powder Window" – any 3-hour-or-longer block where `snowfall > 1cm/hr` AND `wind < 20 km/h`. Add a coloured overlay with label "POWDER WINDOW: 9am-12pm, 8cm forecast".
 - Mobile-friendly: snaps to hour cells on swipe
5. Add the `HourlyForecast` component to every mountain page, between the "Conditions at a

```
glance" section and the "6-day forecast" section. Section header: "Next 48 hours".
```

6. Localise time format to JST for Japan mountains, AEDT for NSW mountains.

7. Run pnpm typecheck. Visit a mountain page and verify the hourly strip renders with real data.

How to test:

- 1 Visit `/yamanouchi/yudanaka/mountains/shiga-kogen` (or whichever Shiga Kogen URL exists)
- 2 Scroll past "Conditions at a glance" → new "Next 48 hours" section appears
- 3 Swipe/scroll the strip → hours scroll smoothly
- 4 Look for the Powder Window overlay if one exists in current data
- 5 Same test on a Snowy Mountains mountain

What to commit: `feat(weather): hourly 48hr forecast + Powder Window detection`

If it goes wrong:

- "API rate-limited" → Open-Meteo is generous (10k/day free) but if hit, move daily + hourly into a single combined fetch
- "Powder Window never highlights" → relax thresholds (snowfall > 0.5cm/hr, wind < 25 km/h) — Australian conditions rarely hit Japow thresholds

SPRINT 2

Stay section build-out

Schema, components, filters, map, affiliate routing — turn the Stay page into a revenue surface.

Part 4 — Sprint 2: Stay section build (Weeks 2–3)

Six prompts that take the Stay section from empty placeholder to fully-functional revenue-generating feature for all 6 base towns.

Prompt 2.1 — Add Stay endpoints to OpenAPI spec [AGENT]

GOAL

Even though stay data is static for now, defining the contract in `lib/api-spec` lets us regenerate the React Query hooks the rest of the app uses, and gives us a clean migration path to API-backed data later.

In the FeelZlike monorepo, `lib/api-spec` is the OpenAPI source of truth. `lib/api-zod` and `lib/api-client-react` are auto-generated from it.

Please:

1. Open `lib/api-spec` and find the existing endpoint definitions to understand the conventions (path naming, response schema patterns, error responses).
2. Add the following endpoints to the spec:
 - GET `/regions/{region}/towns/{town}/stays` – returns `Stay[]`
 - GET `/regions/{region}/towns/{town}/stays/{stayId}` – returns `Stay`
 - GET `/regions/{region}/towns/{town}/eats` – returns `Eat[]`
 - GET `/regions/{region}/towns/{town}/eats/{eatId}` – returns `Eat`
3. Define the `Stay` and `Eat` schemas in the spec to match the curated data structure (see `artifacts/feelzlike/src/data/curated/SCHEMA.md`). Use OpenAPI 3.x discriminated unions (oneOf with discriminator) for the country-specific extra fields.
4. Run the codegen scripts (find them – likely `pnpm -r run codegen`, or `scripts/codegen.sh`, or similar) to regenerate `api-zod` and `api-client-react`.
5. Update `artifacts/feelzlike/src/hooks/useStayEat.ts` to use the newly-generated hooks from `@workspace/api-client-react` INSTEAD of the local static loader. The hooks should resolve from a static handler for now (`artifacts/feelzlike/src/data/index.ts` can stay as the underlying source – just have the new hooks call into it). This way, the same hook signatures work whether data is static or API-backed in future.
6. Run `pnpm typecheck`.

Report what changed in `api-spec` and confirm the regen succeeded.

How to test: `pnpm typecheck` passes; the existing `useStays(town)` calls in any component (none yet) compile against the new hook signatures.

What to commit: `feat(api): add stay/eat endpoints to OpenAPI spec`

Prompt 2.2 — Build the StayCard component [AGENT]

GOAL

A reusable, region-aware listing card that renders both Aussie lodges and Japanese ryokan beautifully without separate components.

Build a StayCard component at `artifacts/feelzlike/src/components/StayCard.tsx` that takes a `Stay` (typed from `artifacts/feelzlike/src/types/stayEat.ts`) and renders:

LAYOUT (mobile-first, responsive):

- Photo carousel at top (Embla, swipe-enabled, fall back to a stylised SVG placeholder if `photos[]` is empty)
- Type badge (RYOKAN / HOTEL / LODGE / APARTMENT / AIRBNB / HOSTEL) with appropriate colour coding
- Name (and `name_local` in smaller text underneath for Japan)
- Short description (1 line, truncated with ...)
- Price band (\$ / \$\$ / \$\$\$ / \$\$\$\$) as filled circles
- Drive-time chips: "→ Thredbo 32min · Perisher 28min" (NSW) or "→ Shiga Kogen 35min · Kita-Shiga 18min" (Japan)
- Region-specific feature badges:
 - NSW: Drying room yes · Ski storage yes · Pet-friendly · Self-contained
 - Japan: Onsen (with type icon) · Tattoo: Allowed/Private/No · Kaiseki dinner · Yukata
- English-spoken indicator for Japan (EN: Fluent / Limited / None)
- "Book on..." button row – one button per `booking_links` entry, branded with the provider's brand colour:
 - Booking.com (#003580) · Agoda (#5392f9) · Airbnb (#FF5A5F) · Expedia (#fcc917) · Hotels.com (#d22630) · Trip.com (#287dfa) · Rakuten (#bf0000) · Jalan (#ff6600)
 - Each button uses the `affiliateLinks` helper (see Prompt 2.5)
 - "Visit official site" as a secondary outline button if `website` is set

INTERACTIONS:

- Whole card is clickable → opens a Sheet/Dialog (Radix) with full-detail view: all photos, full description, every amenity, full hours, full address with copy button, every booking link with prices side-by-side (placeholder "From \$" until we add price scraping), nearest mountain drive times to ALL mountains in region (not just nearest 2)
- "Open in Maps" button using the `lat/lng` (mobile: deep-link to Apple/Google Maps based on user agent)

ACCESSIBILITY:

- All buttons have `aria-labels`
- Photo carousel has keyboard navigation
- Colour-only badges have text equivalents

Use the existing Radix + Tailwind v4 + lucide-react setup. Reference shadow patterns from `artifacts/feelzlike/components.json`.

Add a Storybook-style demo at

`artifacts/feelzlike/src/components/__demos__/StayCard.demo.tsx` that renders 4 cards: a Yamanouchi ryokan (Yorozuya), a Yamanouchi hotel (Hotel Issa-tei), a Jindabyne hotel (Lake Crackenback), a Berridale lodge (Cottonwood Lodge). This lets you preview without wiring routes.

Run `pnpm typecheck`.

How to test: Render the demo file in a temporary route or import it into an existing page; verify all 4 cards display correctly.

What to commit: `feat(stay): region-aware StayCard component with affiliate buttons`

Prompt 2.3 — Build the Stay filter bar [AGENT]

GOAL

Filters that adapt based on whether the user is in an Aussie town or a Japanese town. Different priorities, different available filters.

Build a StayFilterBar component at `artifacts/feelzlike/src/components/StayFilterBar.tsx`.

DESIGN:

- Sticky top bar (sticks below the main app header on scroll)
- Mobile: collapsible into a "Filters" button that opens a Sheet
- Desktop: horizontal pill row with "More filters" overflow

UNIVERSAL FILTERS (always visible):

- Type: chips for Hotel / Ryokan / Lodge / Apartment / Airbnb / Hostel (multi-select; only show types that exist in current dataset)
- Price: \$ / \$\$ / \$\$\$ / \$\$\$\$ (multi-select)
- Sort: dropdown with options:
 - "Drive time to today's #1 mountain" (DEFAULT – uses Today's Call data)
 - "Drive time to nearest mountain"
 - "Price: low to high"
 - "Price: high to low"
 - "Name A-Z"

NSW-ONLY FILTERS (only render when `region === 'snowy_mountains'`):

- Drying room yes
- Ski storage yes
- Pet-friendly yes
- Self-contained (kitchen) yes
- Distance to Thredbo: slider 0-100km
- Distance to Skitube/Perisher: slider 0-100km

JAPAN-ONLY FILTERS (only render when `region === 'yamanouchi'`):

- Onsen: All / Public / Private / Both
- Tattoo policy: All / Allowed / Private only / Not allowed
- Meal plan: All / Half-board (kaiseki) / Breakfast only / None
- English-spoken: All / Fluent / Limited or better
- Walk to Yudanaka Station: slider 0-30 min

PROPS:

- `stays`: Stay[] (current dataset)
- `onChange`: (filters: StayFilters) => void
- `region`: 'snowy_mountains' | 'yamanouchi'

LOGIC:

- Filters are URL-synced (use Wouter's `useLocation` + `URLSearchParams`) so users can share filtered links
- Result count shown inline: "Showing 18 of 25 stays"
- "Clear filters" button when any filter is active
- Active filters shown as chips that can be tapped to remove

Use Radix Toggle, Slider, Select, Sheet primitives.

Run `pnpm typecheck`.

How to test: Add to a Stay page; toggle filters; verify URL updates and result count changes.

What to commit: `feat(stay): region-aware filter bar with URL sync`

Prompt 2.4 — Build the Stay map view [AGENT]

GOAL

A map view showing all stays in the current town as pins, colour-coded by drive time to today's #1 mountain.

Build a StayMap component at `artifacts/feelzlike/src/components/StayMap.tsx` using `react-leaflet` (already a dependency).

FEATURES:

- Centered on the town's coordinates (compute centroid of `stays[]`)
- One marker per stay
- Marker colour by `drive_min` to today's #1 mountain:
 - Green: 0-15 min
 - Yellow: 15-30 min
 - Orange: 30-45 min
 - Red: 45+ min
- Marker icon by type (use `lucide-react` icons rendered to SVG: Hotel, Bed, Tent, Building)
- Click marker → popover with: photo, name, type, price band, drive-time, "View details" button (opens the same Sheet as StayCard click), "Book" button (primary `booking_links` target)
- Toggle between Map view and List view at the page level (we'll add this in 2.6)
- Tile layer: OpenStreetMap (no API key needed), or Mapbox if a token is set in env
- Zoom controls hidden on mobile to save space
- "Locate me" button if user grants geolocation → shows their position + redraws drive-time colours from THEIR location instead of town centre

LAYOUT:

- Full-bleed on mobile (no padding)
- Side-by-side with list on desktop (40% map, 60% list)
- Height: 100vh minus header on mobile, 80vh desktop

PERFORMANCE:

- Use `leaflet.markercluster` if stay count > 50 (npm install needed if not present – flag in summary)
- Lazy-load the map (dynamic import) so it doesn't block initial page render

Filter integration: when `StayFilterBar` updates, the map updates too – same filtered set.

Run `pnpm typecheck`.

How to test: Navigate to Stay page → toggle to map view → see pins coloured by drive time → filters affect what's shown.

What to commit: `feat(stay): map view with drive-time colour coding`

Prompt 2.5 — Affiliate deep-link helper (TypeScript port) [AGENT]

GOAL

Single source of truth for all affiliate URLs. When IDs come through, edit one file and every link updates.

The data package included a JS reference at `artifacts/feelzlike/src/data/curated/affiliate_links.js`. Port this to TypeScript at `artifacts/feelzlike/src/lib/affiliateLinks.ts`.

REQUIREMENTS:

1. Type the `AFFILIATE_IDS` object as `Record<Provider, string | null>` where `Provider` is a string union of all supported providers.
 2. Add a builder per provider (`booking_com`, `agoda`, `airbnb`, `expedia`, `hotels_com`, `trip_com`, `rakuten_travel`, `jalan`, `tripadvisor`, `official`). Each builder takes a `Stay` and returns `string | null`.
 3. Build URLs that:
 - Use the property name + town as search query (so deep-link lands on the right property)
 - Inject the affiliate ID where present, fall back to a clean search URL where not
 - For Japan providers (`rakuten`, `jalan`), use `name_local` (kanji) where available
 - Return null if the provider doesn't serve the stay's country
 4. Export:
 - `buildBookingLinks(stay: Stay): Partial<Record<Provider, string>>`
 - `hasAffiliateId(provider: Provider): boolean`
 - `affiliateStatus(): Record<Provider, 'active' | 'pending'>`
 - `PROVIDER_BRAND_COLOURS: Record<Provider, { bg: string, fg: string }>` – for the StayCard buttons (#003580 white text for Booking, etc.)
 - `PROVIDER_LABELS: Record<Provider, string>` – display names
 5. Add an env-variable override pattern: read affiliate IDs from `import.meta.env.VITE_BOOKING_AFFILIATE_ID` etc., fall back to hardcoded null. Document this in a top-of-file comment so future-you knows to set them in Replit Secrets when ready.
 6. Add unit tests at `artifacts/feelzlike/src/lib/__tests__/affiliateLinks.test.ts`:
 - Builder produces a valid URL for each provider
 - JP-only providers return null for AU stays and vice versa
 - Affiliate ID is correctly injected when present
 - Builder gracefully handles a stay missing optional fields
 7. Wire `affiliateLinks` into `StayCard` (replace any inline URL building from Prompt 2.2).
- Run `pnpm typecheck`.

How to test: Inspect a StayCard in the browser → "Book on Booking.com" link → verify URL has the property name in the query AND your affiliate ID once you set the env var.

What to commit: `feat(affiliate): single-source affiliate URL builder with env override`

Prompt 2.6 — Wire the Stay page (the payoff) [AGENT]

GOAL

Replace the current empty Stay page with the real one — filter bar + map + list, all powered by the curated data.

Replace the current Stay page implementation (likely `artifacts/feelzlike/src/pages/.../stay.tsx` – find it) with the fully-wired version.

LAYOUT:

- Page header: "Stay in {town}" with sub-line: "{count} hand-picked stays – sorted by drive time to today's #1 mountain"
- Today's #1 mountain badge prominently placed: "Today's #1: Yomase Onsen – 91/100" (pull from the existing Today's Call data – the dashboard library already has this)
- StayFilterBar (sticky)
- View toggle: List | Map (use Radix ToggleGroup)
- StayCard list (default) or StayMap (when toggled)
- Existing 8-site search availability box (the one with Booking/Airbnb/Agoda/etc. buttons) keeps its position at bottom – but now it's a SECONDARY option ("Looking for something not listed? Search...")

DATA FLOW:

- `useStays(town)` → all stays for town
- `useTodaysCall(region)` → today's #1 mountain (existing hook from `feelzlike-dashboard`)
- Sort stays by `drive_min_to_each_mountain[todaysWinner]` when `sort = "Drive time to today's #1 mountain"`
- Apply StayFilterBar filters before render

URL STATE:

- Filters in URL params (handled by FilterBar)
- View toggle (list/map) in URL: `?view=map`
- Selected stay (when sheet is open) in URL: `?stay={stayId}` – so opening a card creates a shareable deep-link

EMPTY STATES:

- If filters return zero results: "No stays match these filters. {Clear filters}" instead of empty list
- If a town has zero stays in the underlying data: keep the EmptyStateCard from Prompt 1.3

LOADING STATES:

- Stays are static (instant) but use Suspense boundaries for the map + sheet anyway – keeps the migration to API-backed seamless

REGION GUARD:

- Call `assertProvidersForRegion` equivalent for stays – verify all rendered `stays.region` matches current page region. Prevents the Cooma Coaches class of bug.

Test all 6 town pages:

- `/snowy-mountains/jindabyne/stay` (25 stays)
- `/snowy-mountains/berridale/stay` (12 stays)
- `/snowy-mountains/cooma/stay` (18 stays)
- `/yamanouchi/yudanaka/stay` (21 stays)
- `/yamanouchi/shibu-onsen/stay` (22 stays)
- `/yamanouchi/yomase/stay` (9 stays)

Run `pnpm typecheck`. Take screenshots of each town's stay page if possible (or describe what you see).

How to test:

- 1 All 6 town stay pages render with the right number of stays
- 2 Filter changes update both list and map
- 3 Sort by "drive time to today's #1" actually re-orders based on live Today's Call
- 4 Clicking a card opens the detail sheet
- 5 Booking buttons go to the right URLs (with placeholders since affiliate IDs not yet set)
- 6 URL syncs filters (paste a URL with filters into a fresh tab → filters apply)

What to commit: `feat(stay): launch Stay sections for all 6 base towns`

If it goes wrong:

- "Today's Call data not available" → fall back to "drive time to nearest mountain" sort with a console warning
- "Map performance laggy" → enable marker clustering even at low counts, or virtualise the list

SPRINT 3

Eat section build-out

Restaurant cards, openNow logic, cuisine filters, reservation links — share infra with Stay.

Part 5 — Sprint 3: Eat section build (Weeks 4–5)

Five prompts that take Eat from empty to fully functional. Reuses patterns from Sprint 2 — faster than Sprint 2 because the heavy lifting (filter bar pattern, map pattern, region-aware sheet) is already in place.

Prompt 3.1 — Add Eat endpoints to OpenAPI spec [AGENT]

GOAL

Same flow as Stay endpoints — define in spec, regen, ready for static-or-API consumption.

The Stay endpoints were added to lib/api-spec in Prompt 2.1. Do the same for Eat:

1. Add to lib/api-spec:
 - GET /regions/{region}/towns/{town}/eats — returns Eat[]
 - GET /regions/{region}/towns/{town}/eats/{eatId} — returns Eat
2. Define the Eat schema in the spec, mirroring artifacts/feelzlike/src/data/curated/SCHEMA.md. Discriminated union on country: 'AU' adds apres_ski/takeaway/groceries; 'JP' adds vegetarian_friendly/kid_friendly. Always include: cuisine[], hours{by_day}, last_order_time, reservation enum, payment enum, english_menu enum, signature_dishes[].
3. Regenerate api-zod and api-client-react.
4. Update artifacts/feelzlike/src/hooks/useStayEat.ts useEats(town) to use the generated hook.
5. pnpm typecheck.

What to commit: feat(api): add eat endpoints to OpenAPI spec

Prompt 3.2 — Build the EatCard component [AGENT]

GOAL

Compact, scannable card for restaurants/izakaya/cafes/bars/essentials. Different visual treatment than StayCard — eats are higher-density (people scan many quickly).

Build an EatCard component at artifacts/feelzlike/src/components/EatCard.tsx.

- LAYOUT (denser than StayCard — these are scanned quickly):
- Compact horizontal layout on desktop (photo left 30%, content right 70%)
 - Stacked on mobile (photo top, content bottom)
 - Photo (single hero image, fall back to type-iconographic placeholder)
 - Type chip top-left (RAMEN / IZAKAYA / CAFE / RESTAURANT / BAR / BAKERY / PUB / GROCERY /

```
FUEL)
- Name + name_local (Japan)
- Cuisine chips (max 3)
- Price band ($ / $$ / $$$)
- "Open now" / "Closed" pill — REAL-TIME based on hours{} and current time in town's
  timezone
  · Green "Open · Last order 9pm" when open
  · Yellow "Closing in 30min" when closing soon
  · Red "Closed · Opens 5:30am" when closed
  · Grey "Hours unverified" when hours data is null
- Critical badges (compact icon row):
  · [cash] Cash only (Japan)
  · [menu] EN menu / [photo] Picture menu / no JP only (Japan)
  · [veg] Veg-friendly / Limited (Japan)
  · [ski] Après-ski (NSW)
  · [takeaway] Takeaway
  · [grocery] Groceries (for IGA/supermarket entries)
- Signature dishes: shown as small italicised quote ("Famous for: Shinshu soba, oyaki")
- Action row:
  · "Reserve" button (only if reservation_link present — Tabelog / Hotpepper / OpenTable /
    phone)
  · "Call" button (tel: link with phone)
  · "Directions" button (maps deep-link)

INTERACTIONS:
- Click → detail Sheet with full hours table, full address, cash-or-cards, full
  description, all signature dishes, all photos, distance from staying-in town

RUN pnpm typecheck.
```

How to test: Render in a demo file with 6 examples (one of each type from each region).

What to commit: feat(eat): EatCard component with open-now indicator

Prompt 3.3 — Build the Eat filter bar [AGENT]

GOAL

Filters tuned for restaurant discovery — not a copy of Stay filters.

Build an EatFilterBar at artifacts/feelzlike/src/components/EatFilterBar.tsx.

UNIVERSAL FILTERS:

- Type: chips for Restaurant / Izakaya / Cafe / Bar / Pub / Ramen / Soba / Bakery / Grocery / Fuel (multi-select; only show types present)
- Cuisine: multi-select dropdown (cuisine[] union of all eats in current town)
- Price: \$ / \$\$ / \$\$\$
- "Open now" toggle (uses hours data + town timezone)
- Sort: Distance / Open now / Price / Name

NSW-ONLY:

- Après-ski (open after 5pm + alcohol)
- Takeaway available
- Has groceries (helps people find IGA, bakery, bottle-o quickly)

JAPAN-ONLY:

```
- English support: Any / EN menu / Picture menu / Limited or better
- Payment: Any / Cards accepted (excludes cash-only)
- Vegetarian-friendly: Any / Yes / Limited (be honest – rural Nagano has very few
  vegetarian places)
- Kid-friendly

URL SYNC same pattern as StayFilterBar.

REUSE the FilterBar shell from Stay (extract a generic FilterBar primitive if it makes
  sense – look for opportunities to DRY).

Run pnpm typecheck.
```

What to commit: `feat(eat): region-aware Eat filter bar`

Prompt 3.4 — "Open now" logic with timezone awareness [AGENT]

GOAL

Bullet-proof open/closed calculation that handles timezones, overnight hours (bars closing at 2am), holidays, and missing data.

Build `artifacts/feelzlike/src/lib/openNow.ts`:

```
FUNCTION: isOpenNow(eat: Eat): { status: 'open' | 'closing_soon' | 'closed' | 'unknown',
  message: string, nextChange?: Date }
```

LOGIC:

- Determine the eat's timezone:
 - country === 'AU' → 'Australia/Sydney'
 - country === 'JP' → 'Asia/Tokyo'
- Get current time in that timezone (use `Intl.DateTimeFormat` or `date-fns-tz`)
- Get today's hours from `eat.hours[currentDayName]`
- Handle missing data → 'unknown' with message 'Hours unverified'
- Handle 'closed' for the day → 'closed' with message like 'Closed today · Opens tomorrow 11am'
- Handle overnight hours (e.g. bar 5pm-2am): if current time > close time of yesterday but < end-of-yesterday's-overnight-window, still open
- If open and within 30 min of close → 'closing_soon' with message 'Closing in {N} min'
- If open → 'open' with message 'Open · Last order {lastOrder}'
- Compute `nextChange` (when the status will next flip) for use in re-render scheduling

USE `last_order_time` when set (always before close time for kitchens), `close_time` otherwise.

UNIT TESTS at `artifacts/feelzlike/src/lib/__tests__/openNow.test.ts` covering:

- Basic open/closed
- Timezone handling (test with mocked dates)
- Overnight hours (bar open until 2am)
- Closing soon threshold
- Unknown hours
- Day boundary edge cases (11:59pm Sunday → Monday hours)

WIRE into `EatCard` so the badge updates correctly.

Add a `useEffect` that re-renders the badge at `nextChange` so a card that's "Open" naturally flips to "Closed" without page refresh.

Run `pnpm typecheck` and the tests.

What to commit: `feat(eat): timezone-aware open-now logic`

Prompt 3.5 — Wire the Eat page + reservation deep-links [AGENT]

GOAL

Replace the empty Eat page with the real one. Plus the reservation deep-link layer (Tabelog/Hotpepper for JP, OpenTable/TheFork for AU, fallback to tel:).

Build `artifacts/feelzlike/src/lib/reservationLinks.ts`:

```
FUNCTION: buildReservationLinks(eat: Eat): { primary: { provider: string, url: string, label: string }, all: Array<{provider, url, label}> }
```

LOGIC:

- If `reservation === 'not_accepted'` → primary is null, label "Walk-in only"
- If `reservation === 'not_needed'` → primary is null, label "No reservation needed"
- If `reservation_link` is set → primary = direct deep-link
 - Tabelog (`tabelog.com`) → "Reserve on Tabelog"
 - Hotpepper (`hotpepper.jp`) → "Reserve on Hotpepper"
 - OpenTable (`opentable.com`) → "Reserve on OpenTable"
 - TheFork → "Reserve on TheFork"
 - Direct restaurant URL → "Book on official site"
- If only phone is set → primary = ``tel:${phone}``, label "Call to reserve"
- Always include all available options in ``all``

Then wire the Eat page (`artifacts/feelzlike/src/pages/.../eat.tsx` – find it):

LAYOUT same shape as Stay page:

- "Eat in {town} · {count} curated spots" header
- "Open now" quick filter chip prominently placed (one tap to filter)
- `EatFilterBar` (sticky)
- View toggle: List | Map (reuse Stay map but for eats – mark up the `StayMap` so it can take an `items` prop and render eats too, or build `EatMap`)
- `EatCard` list / map view

DATA FLOW:

- `useEats(town)`
- Sort by "Open now" by default (open spots first, then closing soon, then closed)
- Apply filters
- Re-render every 60 seconds or when `openNow` `nextChange` fires

EMPTY STATES:

- All filters applied + zero results: "No spots match. {Clear filters}"
- Town has no eats: `EmptyStateCard` from Prompt 1.3 with "Suggest a spot" CTA

REGION GUARD: same as Stay – assert all `eats.region` matches page region.

Test all 6 town eat pages:

```
- /snowy-mountains/jindabyne/eat (30 eats)
- /snowy-mountains/berridale/eat (13 eats)
- /snowy-mountains/cooma/eat (26 eats)
- /yamanouchi/yudanaka/eat (25 eats)
- /yamanouchi/shibu-onsen/eat (16 eats)
- /yamanouchi/yomase/eat (11 eats)
```

Run `pnpm typecheck`.

How to test:

- 1 All 6 town eat pages render
- 2 "Open now" badges are accurate for current time in Sydney/Tokyo
- 3 Filter to "Open now + cards accepted + EN menu" in Yudanaka — should narrow significantly
- 4 Reservation buttons go to correct URLs (Tabelog deep-links work)
- 5 `tel:` links open phone dialer on mobile

What to commit: `feat(eat): launch Eat sections for all 6 base towns + reservation deep-links`

SPRINT 4

The differentiators

AI personalization, Powder Window, webcams, wind-hold prediction, Powder Factor score.

Part 6 — Sprint 4: Differentiator features (Weeks 6–7)

Five prompts for the features that make people tell their friends. After this sprint, FeelZlike has features no other ski app in the AU/JP market has.

Prompt 4.1 — AI "Today's Call" personalisation [AGENT]

GOAL

The current Today's Call ranks mountains by raw weather metrics. Personalise it: a beginner gets different rankings than an expert; a powder hunter weights snow heavily; a family weights crowds and beginner terrain.

The existing "Today's Call" feature (in lib/feelzlike-dashboard) ranks mountains by a score based on snow, wind, temperature, visibility. Add personalisation so the score weights shift based on the user's profile.

Step 1 — User profile:

- Add a UserProfile schema at artifacts/feelzlike/src/types/profile.ts:

```
{ skill_level: 'beginner'|'intermediate'|'advanced'|'expert',
  discipline: 'ski'|'snowboard'|'both',
  priorities: Array<'powder'|'groomers'|'park'|'backcountry'|'family'|'value'|'crowds'>,
  risk_tolerance: 'low'|'medium'|'high' }
```
- Persist to localStorage; first-time users see a 30-second 4-question onboarding sheet to set this up
- Add a "Profile" link in the sidebar that opens a Sheet to edit anytime

Step 2 — Weighted scoring:

- Default weights (current state): snow 30, wind 25, temp 20, visibility 15, crowds 10
- Profile modifiers (multiplicative):
 - skill_level === 'beginner' → boost mountains tagged 'beginner_friendly' by +20, penalise 'expert_only' by -30
 - skill_level === 'expert' → opposite
 - priorities includes 'powder' → snow weight x2
 - priorities includes 'groomers' → recent_groom weight added (need new data field)
 - priorities includes 'family' → boost mountains with kids_lessons, penalise long drives
 - priorities includes 'crowds' → factor in day-of-week + holiday + storm-day proxy for crowd estimate
 - risk_tolerance === 'low' → harshly penalise wind > 35km/h and visibility < 200m

Step 3 — UI changes:

- Today's Call header: "Today's call · personalised for you" with a subtle "[edit profile]" link
- Each ranked card shows the weights applied: "Why this ranks #1 for you: heavy fresh snow + low wind + your powder priority"
- A "Show non-personalised" toggle for comparison

Step 4 — Mountain data tags:

- Extend the mountain data with: beginner_friendly: bool, expert_only: bool, kids_lessons: bool, terrain_park: bool, backcountry_access: bool — populate for the existing mountains in both regions

Step 5 – Tests for the scoring function with mocked profiles.

Run `pnpm typecheck`.

What to commit: `feat(todays-call): user-profile-weighted personalisation`

Prompt 4.2 — Powder Window overlay refinement [ASSISTANT]

GOAL

Polish the Powder Window detection added in Sprint 1. Right now it highlights any 3-hour block; make it smarter and more visually compelling.

Open `artifacts/feelzlike/src/components/HourlyForecast.tsx` and paste:

Refine the Powder Window logic and UI:

1. Detection improvements:

- A "Powder Window" requires: snowfall > 1cm/hr (relax to 0.5 for AU), wind < 20 km/h, visibility > 500m, temperature < +2°C
- Multiple windows per day are possible – detect them all
- Rank windows by quality: total snowfall over the window × inverse wind × duration
- Tag each: GOLD (best), SILVER, BRONZE windows

2. Visual:

- Each window is overlaid on the hourly strip with a coloured band: gold/silver/bronze
- Top of the band shows: "POWDER WINDOW · 9am-12pm · 8cm forecast · 4 km/h wind"
- Animate a subtle shimmer on GOLD windows (Framer Motion)
- Click the band → opens a detail card explaining the metrics

3. Cross-mountain summary:

- On the region overview page, show "Best Powder Window today: 9-12pm at Yakebitaiyama (GOLD)" with one tap to that mountain's hourly view

4. Mountain pages get a "Powder Calendar" – next 7 days, colour-coded by best window quality each day. Glanceable forecast for trip-planners.

Run `pnpm typecheck`.

What to commit: `feat(weather): Powder Window quality grading + Powder Calendar`

Prompt 4.3 — Webcam embed component [AGENT]

GOAL

Embed live resort webcams directly in mountain pages — don't make users click out.

Build `artifacts/feelzlike/src/components/MountainWebcams.tsx` and a data file at `artifacts/feelzlike/src/data/webcams.ts`.

Webcams data (research from official resort sites – verify each URL works):


```
YAMANOUCHI (Shiga Kogen has many; pick 1-2 per sub-resort):
- Yakebitaiyama base + summit
- Okushiga Kogen
- Ichinose Family
- Yokoteyama
- Sun Valley / Giant
- Ryuoo (the SORA terrace cam is iconic)
- Kita-Shiga Kogen
- Yomase Onsen
- Madarao Kogen
- Tangram
- Togari
- Nozawa Onsen (multi-cam – pick the gondola top + village square)

SNOWY MOUNTAINS:
- Thredbo: top of Kosciuszko Express, Friday Flat, village
- Perisher: Front Valley, Blue Cow, Smiggin Holes, Mt P
- Charlotte Pass: village + main run
- Selwyn: main lift

WEBCAM TYPES TO HANDLE:
- Direct image URL that updates every N seconds → embed as <img> with timestamp +
  auto-refresh
- HLS/m3u8 video stream → use video.js or hls.js (npm install)
- iframe-only embed → use a sandboxed <iframe>
- Snapshot-only (no live) → embed with "Last snapshot {timestamp}" caption
- Mark unverified webcams clearly: "Source: Resort Official Site (last verified {date})"

UI:
- Grid of webcam thumbnails, click to expand to fullscreen
- Auto-rotate carousel mode (cycles every 8 sec) for ambient display
- Show timestamp + "auto-refresh" indicator
- Fall back to "View on resort site" link if embed fails

Wire into mountain pages – replace the existing "external link to webcams" with the
  embedded version.

Run npm typecheck.
```

What to commit: feat(webcams): embedded resort webcams on mountain pages

If it goes wrong: Many resort webcams have CORS or hotlink protection. If embed fails, gracefully fall back to "View on resort site" — don't ship broken images.

Prompt 4.4 — Wind-hold prediction badges for lifts [AGENT]

GOAL

Perisher and Thredbo lose lifts to wind constantly. Predict which lifts will spin tomorrow based on forecast wind speeds at lift altitudes. This is a feature no competitor has.

```
Build artifacts/feelzlike/src/lib/windHoldPrediction.ts.
```

```
DATA NEEDED — per lift, define:
```

```
{
  id, name, mountain, base_elevation, top_elevation,
  exposure: 'sheltered'|'moderate'|'exposed'|'highly_exposed',
  wind_hold_threshold_kmh: number, // typical operating threshold
  type: 'gondola'|'detachable'|'fixed_grip_chair'|'t-bar'|'rope_tow' // gondolas hold
    higher; t-bars run in worse weather
}
```

Seed data for the lifts at:

- Thredbo (Kosciuszko Express, Snowgums, Cruiser, etc.)
- Perisher (Front Valley Quad, Mt Perisher, Blue Cow, V8, etc.)
- Charlotte Pass
- The major Shiga Kogen sub-resorts (top gondolas, key chairs)

Source from: resort official lift maps + historical wind-hold data (the resorts often publish "lift status" pages – check 5+ days history if available).

PREDICTION FUNCTION: predictLiftStatus(lift, hourlyWindForecast): { status: 'likely_open'|'possible_hold'|'likely_held', confidence: 0-1, reason: string }

LOGIC:

- Take hourly wind + gust forecast at the lift's TOP elevation (not summit) – interpolate from Open-Meteo's elevation-based data
- Apply exposure multiplier: highly_exposed lifts hold at lower thresholds
- 'likely_open' if forecast wind+gusts < 70% of threshold
- 'possible_hold' if 70-110% of threshold
- 'likely_held' if > 110% of threshold

UI:

- On mountain pages, lift status section shows each lift with a coloured badge: green (open), yellow (possible hold), red (likely held)
- Hover/tap → tooltip "Likely held: forecast 60km/h gusts at top, threshold 50km/h"
- Compare to live lift status from resort APIs where available – track prediction accuracy over time (log to Sentry breadcrumbs for our own learning)

DISCLAIMER COPY: "Predictions only – lift operation depends on resort decisions. Always check resort site before driving."

Run `pnpm typecheck`.

What to commit: `feat(lifts): wind-hold prediction with confidence scoring`

Prompt 4.5 — Snow quality (powder factor) score [ASSISTANT]

GOAL

Snow depth alone is misleading. A 30cm dump that fell at -2°C with low humidity skis very differently than 30cm that fell wet at +1°C. Build a "Powder Factor" score.

Open the existing scoring logic in `lib/feelzlike-dashboard` and paste:

```
Add a Powder Factor sub-score to mountain weather:

POWDER FACTOR (0-100) is computed from recent 24/48hr conditions:
- Snowfall amount (more = better, asymptotic at 50cm)
- Snow temperature at fall: colder = drier = better. Optimal -8°C to -3°C
```

- Humidity at time of fall: lower = lighter snow
- Wind during fall: high wind compacts snow & creates wind slab risk = penalty
- Days since last melt cycle (temp > +2°C): more days = stale snow = penalty
- Recent rain: any rain = harsh penalty (creates ice layer)

DERIVED LABELS:

- 90-100: "Bottomless Japow"
- 75-89: "Premium powder"
- 60-74: "Soft and fast"
- 45-59: "Decent – no ice"
- 30-44: "Hardpack – variable"
- 0-29: "Survival skiing"

UI:

- Show Powder Factor on every mountain page next to depth and freezing level
- A small "?" icon explains the formula
- Add to Today's Call as a sub-column

This will rank Yakebitaiyama (high cold dry) above Yomase Onsen (lower altitude, warmer) on a marginal day even when total depth is similar.

Run pnpm typecheck.

What to commit: feat(weather): Powder Factor score with quality labels

SPRINT 5

Alerts & subscriptions

Email + web push + cron triggers — bring people back when conditions are right.

Part 7 — Sprint 5: Powder alert subscriptions (Weeks 8–9)

The infrastructure for storm tracking already exists in v0.3 (the Storm Tracker on the Alerts page). This sprint turns it into a subscription product — email + web push.

Prompt 5.1 — Email subscription form + storage [AGENT]

GOAL

Let users subscribe to powder alerts by email. Build the subscription UI, the storage layer, and basic preference management.

Build the email subscription system for powder alerts.

DATA MODEL:

- Use the existing lib/db package (whatever DB it wraps – Postgres? SQLite? Replit DB? Check the setup first)
- Create a Subscriber table:

```
{ id, email, created_at, verified_at?, unsubscribed_at?,
  regions: RegionSlug[],           // which regions
  mountains: MountainSlug[],       // optional: specific mountains
  snowfall_threshold_cm: number,    // alert when forecast >= X cm in 24/48hr
  horizon_hours: 24|48|72,         // how far ahead to alert
  delivery: 'email'|'push'|'both',
  timezone: string,
  profile_token?: string // link to UserProfile if logged in / browser-fingerprinted
}
```

API ENDPOINTS (add to lib/api-spec, regen):

- POST /alerts/subscribe – body: email, regions, threshold etc. Returns subscriber id + verification link
- GET /alerts/verify?token=... – confirms email
- GET /alerts/manage?token=... – returns subscriber preferences
- PUT /alerts/manage?token=... – update preferences
- POST /alerts/unsubscribe?token=... – soft-delete

EMAIL VERIFICATION:

- Use double-opt-in (legally required in many regions; protects deliverability)
- Send verification email via Resend, Postmark, or SendGrid (check what's already in @workspace/integrations or env vars; if nothing, add Resend – simplest, generous free tier, AU-friendly)
- Email template lives in artifacts/feelzlike/src/emails/ or a dedicated package – use react-email if not already present
- Branded template: navy header with logo, body in serif, "Confirm your subscription" CTA

UI:

- "Subscribe to powder alerts" section on the existing Alerts page
- Form fields: email, regions multi-select (default current region), threshold (slider 5-50cm), horizon (24/48/72hr buttons), delivery method
- After submit: "Check your email to confirm" toast
- Verification page: "You're in. Powder incoming." with link back to app

- Manage preferences page accessible via the link in every alert email

PRIVACY:

- Plain English consent: "We'll only email you when forecast snowfall meets your threshold. Unsubscribe in one click."
- Privacy policy link (placeholder URL)

Run `pnpm typecheck`.

What to commit: `feat(alerts): email subscription with double-opt-in`

Prompt 5.2 — Web push notification setup [AGENT]

GOAL

Let users opt into browser push notifications for the same alerts. Service worker, push subscription, VAPID keys.

Add web push notifications.

SERVICE WORKER:

- Create `artifacts/feelzlike/public/sw.js` – handles push events, displays notifications, deep-links into the app on click
- Register the service worker in `artifacts/feelzlike/src/main.tsx`
- Make sure Vite's PWA setup doesn't conflict (we'll add proper PWA in Sprint 6 – this SW is alert-specific for now or we structure it for later PWA reuse)

VAPID KEYS:

- Generate VAPID public/private key pair (can use web-push npm package: `web-push generate-vapid-keys`)
- Store private key in Replit Secrets as `VAPID_PRIVATE_KEY`
- Store public key in app env as `VITE_VAPID_PUBLIC_KEY`
- Document in README

FLOW:

1. User clicks "Enable push notifications" in Alerts subscription form
2. Browser permission prompt (handle denied gracefully – show "Enable in browser settings" copy)
3. On grant, get `PushSubscription` via service worker registration
4. POST to `/alerts/subscribe` with `delivery: 'push'` and the subscription object
5. Server stores subscription endpoint + p256dh + auth keys

SERVER-SIDE PUSH:

- Use the web-push npm package on the api-server side
- Add an endpoint `POST /alerts/test-push?token=...` to send a test push for development
- The actual production push trigger is in Prompt 5.3

NOTIFICATION FORMAT:

- Title: "* Powder incoming at Yakebitaiyama"
- Body: "30cm forecast in next 24hr. Wind 8 km/h. Tap to plan your day."
- Icon: FeelZlike logo
- Click → opens the app to the relevant mountain page

iOS NOTE:

- iOS Safari supports web push only when the site is added to home screen as a PWA

- Detect Safari iOS, show "Add FeelZlike to home screen to enable push" instead of the standard prompt

Run `pnpm typecheck`.

What to commit: `feat(alerts): web push notifications + service worker`

Prompt 5.3 — Alert trigger logic (cron + threshold rules) [AGENT]

GOAL

The scheduled job that checks forecasts, matches against subscribers, fires emails and pushes.

Build the alert evaluation + dispatch system.

WHERE THIS RUNS:

- The api-server is the right home – it has DB access and runs server-side
- Use either `node-cron` OR a Replit Deployments scheduled task (whichever is conventional in the existing setup – check)
- Schedule: every 3 hours (forecasts don't change minute-by-minute; 3hr is a good balance of freshness vs server cost vs notification fatigue)

EVALUATION LOOP:

1. Pull all verified, non-unsubscribed subscribers
2. For each unique (region, threshold, horizon) combination, fetch the latest mountain forecasts (cache to avoid hitting Open-Meteo per subscriber)
3. For each subscriber, find mountains in their selected regions/specific mountains where forecast snowfall in the next `horizon_hours` $\geq$ their threshold
4. Match against a "last sent" log to avoid spamming (don't send same alert about same event twice – define "same event" as same mountain + same forecast snow window within last 24hr)
5. Build personalised alert payload per subscriber

DISPATCH:

- delivery includes 'email' → send via Resend with template "Powder alert: {mountain}"
- delivery includes 'push' → send via web-push to all that subscriber's push subscriptions

EMAIL TEMPLATE:

- Subject: `"* {topMountain} powder alert: {amount}cm forecast"`
- Body: top 3 mountains in their regions with snow forecast, drive times from the user's saved town (if known), link to Today's Call, link to manage preferences
- Always include unsubscribe link

LOGGING:

- Log every dispatch to Sentry breadcrumbs (`subscriber_id`, `alert_type`, `mountain`, `snow_forecast`)
- Daily summary metric: alerts sent, click-through rate (track via UTM-tagged links)

RATE LIMIT:

- Max 1 alert per subscriber per 12 hours regardless of triggers
- Honour quiet hours (no alerts 10pm-6am in subscriber's timezone) – except for "extreme" events (>50 cm forecast)

Run `pnpm typecheck`.

What to commit: `feat(alerts): scheduled alert evaluation + dispatch`

Prompt 5.4 — Subscription management page [AGENT]

GOAL

Users can manage their subscription via a tokenised link in every email. No login required (one-click access from email).

Build the subscription management page at `/alerts/manage?token=...`

LAYOUT:

- Header: "Your alerts · {email}"
- Sections:
 - Regions (toggle on/off)
 - Mountains (multi-select within their regions; "All mountains" default)
 - Snowfall threshold (slider 5-50cm with live preview "You'd have got 3 alerts in the last 30 days at this threshold")
 - Horizon (24/48/72hr buttons)
 - Delivery (email / push / both checkboxes)
 - Quiet hours (timezone-aware)
- "Save changes" button
- Big red "Unsubscribe from all" link at the bottom

DATA:

- Token (signed/HMAC'd) maps to subscriber id
- All changes immediate (no email re-confirmation)
- Unsubscribe shows a goodbye page: "You're unsubscribed. Resubscribe anytime at `feelzlike.com/alerts`. We'd love your feedback: {short form}."

ANALYTICS:

- Track preference changes (anonymised)
- Track unsubscribes by reason (optional radio: too many alerts, threshold wrong, no longer relevant, other)

Run `pnpm typecheck`.

What to commit: `feat(alerts): subscription management page with one-click access`

SPRINT 6

Polish & launch

Homepage redesign, PWA install, SEO, error tracking. Ship it.

Part 8 — Sprint 6: Polish & launch prep (Weeks 10–12)

Four prompts to take the app from "feature-complete" to "ready to put your name on."

Prompt 6.1 — Homepage redesign with hero imagery + live preview [AGENT]

GOAL

The current homepage is bare — logo, headline, three region cards. It doesn't reflect the depth inside. Redesign it to make a first-time visitor immediately understand the value and want to explore.

Redesign artifacts/feelzlike/src/pages/index.tsx (or equivalent home route).

DESIGN GOALS:

- First-time visitor understands "this is THE ski conditions app for AU and JP" within 2 seconds
- Existing v0.3 brand (deep navy, sky-blue accent, bold typographic hero) is the foundation
 - don't redesign the brand, build on it
- Mobile-first

NEW STRUCTURE:

1. Hero section (full-viewport on first load):
 - Background: full-bleed photo of Japow trees OR Aussie alpine — use a curated set of 4-6 hero photos that rotate per visit (each photo credits photographer in small text bottom-right)
 - Logo top-left, language toggle top-right
 - Headline: "What does the mountain feel like today?" (existing)
 - Sub: "Hand-built for skiers and snowboarders in NSW and Nagano. Weather, stays, eats, alerts — all in one place."
 - Two big CTA cards: "Explore Snowy Mountains" / "Explore Yamanouchi" — each with mini live data preview ("3 mountains scoring above 80 right now")
2. Today's call preview (live data, both regions):
 - Two side-by-side cards: top mountain in each region with score, snow forecast, drive time from main base town
 - "See full ranking →" link
3. Powder tracker:
 - Live storm map (mini version of the Radar page)
 - "Current alerts" badge if any active
4. Stay + Eat preview:
 - "228 hand-picked stays and eats across 6 base towns"
 - Stack of 3 stay cards + 3 eat cards (use existing components, smaller variant)
5. Why FeelZlike (the value prop, only shown to first-time visitors via cookie check):
 - 3 columns: "Live conditions you can trust" / "Stays curated for skiers" / "Powder alerts" — each with one-line copy + icon

6. Footer:

- Privacy, Terms, Contact, Photo credits, "Built with care in Australia"

ASSETS NEEDED:

- 4-6 hero photos: source from Unsplash with proper attribution (search "Japow", "Hakuba", "Thredbo snow", "Perisher backcountry"), OR commission your own / use friends' photos
- For now, fall back to a beautifully gradient-overlaid stock photo with a TODO comment

PERFORMANCE:

- Hero photo lazy-load with low-quality placeholder
- Live data sections use Suspense – page shell loads immediately
- Lighthouse target: 90+ Performance, 100 Accessibility

Run pnpm typecheck. Take a screenshot of the new homepage.

What to commit: feat(home): redesigned homepage with hero imagery + live previews

Prompt 6.2 — Mobile PWA setup [AGENT]

GOAL

Make FeelZlike installable as a home-screen app on iOS and Android. Critical for retention and re-engagement (people who add to home screen open ~3x more often).

Convert FeelZlike into a Progressive Web App.

DEPENDENCIES:

- Add vite-plugin-pwa to artifacts/feelzlike (pnpm add -D vite-plugin-pwa workbox-window)
- Configure in vite.config.ts: registerType 'autoUpdate', includeAssets, manifest config

MANIFEST (artifacts/feelzlike/public/manifest.json or via plugin):

```
{
  name: "FeelZlike – Ski & Snowboard Conditions",
  short_name: "FeelZlike",
  description: "What does the mountain feel like today?",
  theme_color: "#0a1f44", // your deep navy
  background_color: "#ffffff",
  display: "standalone",
  orientation: "portrait",
  start_url: "/",
  icons: [
    { src: "/icons/icon-192.png", sizes: "192x192", type: "image/png" },
    { src: "/icons/icon-512.png", sizes: "512x512", type: "image/png" },
    { src: "/icons/icon-512-maskable.png", sizes: "512x512", type: "image/png", purpose:
      "maskable" },
    { src: "/icons/apple-touch-icon.png", sizes: "180x180", type: "image/png" }
  ],
  categories: ["weather", "travel", "sports"]
}
```

Generate icons from the existing FeelZlike logo SVG – use a tool like pwa-asset-generator or do it manually in Figma. All sizes needed: 192, 512, 512-maskable, apple-touch-icon-180.

SERVICE WORKER:

- Reuse the SW from Prompt 5.2 (alerts) – extend it with workbox caching strategies:
 - Cache-first for static assets (fonts, icons, JS chunks)
 - Network-first for API calls (Today's Call, weather)
 - Stale-while-revalidate for the Stay/Eat data (changes occasionally)

INSTALL PROMPT:

- Build artifacts/feelzlike/src/components/InstallPrompt.tsx
- Listens for beforeinstallprompt event
- Shows a non-intrusive banner after the user has visited 3+ times: "Add FeelZlike to your home screen – never miss a powder day"
- Dismissible; respects dismissal for 30 days
- iOS-specific copy (since iOS doesn't fire beforeinstallprompt): "Tap Share → Add to Home Screen"

OFFLINE:

- Show a friendly "You're offline – showing last cached conditions" banner
- Last cached weather data still renders (not a blank page)
- Booking links + maps require online – graceful error if tapped offline

iOS QUIRKS:

- Add apple-mobile-web-app-capable meta tag
- Status bar style
- Splash screens (generate with pwa-asset-generator)

Run `pnpm build`. Test installability in Chrome DevTools → Application → Manifest. On a real iPhone via Safari → Share → Add to Home Screen → confirm icon, splash, standalone mode.

Run `pnpm typecheck`.

What to commit: `feat(pwa): installable PWA with offline support`

Prompt 6.3 — SEO meta tags + sitemap [AGENT]

GOAL

Stay/eat pages should rank for "ryokan in Yudanaka", "where to stay in Jindabyne", etc. Right now there's nothing for Google.

Add comprehensive SEO to FeelZlike.

PER-PAGE META TAGS:

- Use `react-helmet-async` (or whatever the existing pattern is – check if any meta tags are set anywhere) – install if needed
- Build a `<PageMeta />` component that wraps Helmet, takes typed props

PAGE-SPECIFIC TEMPLATES:

- Homepage: "FeelZlike – Live ski conditions, stays & eats for NSW + Nagano"
- Region overview: "{Region} Ski Conditions · Live Today's Call · FeelZlike"
- Town page: "Where to stay & eat in {Town} · Live mountain conditions · FeelZlike"
- Mountain page: "{Mountain} live conditions, snow report, hourly forecast · FeelZlike"
- Stay listing: "Stays in {Town} · {count} hand-picked options for skiers · FeelZlike"
- Eat listing: "Eat in {Town} · {count} curated restaurants and bars · FeelZlike"
- Stay/Eat detail (when sheet has its own URL): "{name} · {town} · FeelZlike review"

OG TAGS for social sharing:

- og:title, og:description, og:image (per-region hero photo or per-property hero)
- twitter:card summary_large_image
- og:site_name FeelZlike

STRUCTURED DATA (JSON-LD):

- LodgingBusiness schema for each Stay (name, address, geo, priceRange, image, amenityFeature[])
- Restaurant schema for each Eat (cuisine, openingHours, acceptsReservations, priceRange)
- Place schema for each mountain
- This unlocks rich Google results (price, hours, ratings)

SITEMAP:

- Generate at build time via vite-plugin-sitemap or a custom build step
- Include every region, town, mountain, stay, and eat page
- Submit to Google Search Console (manual step – document in deploy notes)

ROBOTS.TXT:

- Allow all crawlers
- Reference sitemap

CANONICAL URLs:

- Set <link rel="canonical"> on every page (especially important once we have filter URL params – point canonicals at the unfiltered version)

PERFORMANCE for SEO:

- Lighthouse SEO score target: 100
- Core Web Vitals: green across LCP, FID, CLS

Run `pnpm build`, then crawl the `dist/` output with a tool like `sitemap-generator-cli` to verify all URLs are picked up.

Run `pnpm typecheck`.

What to commit: `feat(seo): full meta tags, structured data, sitemap`

Prompt 6.4 — Sentry monitoring for new sections + analytics [AGENT]

GOAL

Production telemetry. We need to know when things break, what users actually do, and which booking links convert.

Extend Sentry coverage and add product analytics.

SENTRY:

- Sentry React is already wired (per package.json)
- Audit: ensure error boundaries wrap each major page section (Stay, Eat, Mountain, Today's Call, Alerts)
- Add user context: anonymised `profile_token`, `region`, `town`
- Add breadcrumbs for: filter changes, view toggles (list/map), affiliate clicks, alert subscription events
- Set up Performance Monitoring for: page load, API call duration, map render time
- Configure release tagging (existing build script does the SHA)

PRODUCT ANALYTICS:

- Add Plausible or PostHog (privacy-respecting, GDPR-friendly – choose Plausible for simplicity, PostHog for power)
- Events to track:
 - page_view (auto)
 - region_select, town_select, mountain_select
 - todays_call_view (which mountain they tapped)
 - stay_view (which stay opened the detail sheet)
 - eat_view
 - affiliate_click (provider, stay_id, town) – KEY METRIC for revenue
 - reservation_click (provider, eat_id)
 - alert_subscribe (region, threshold, delivery)
 - alert_unsubscribe (with reason if given)
 - profile_set (skill_level, priorities)
 - install_prompt_shown / install_prompt_dismissed / install_prompt_accepted
 - pwa_used (true if standalone mode detected on load)

DASHBOARDS:

- Create saved dashboards for:
 - Daily active users by region
 - Top mountains tapped from Today's Call
 - Affiliate click-through funnel (Stay view → affiliate click → estimated booking)
 - Alert performance (sent → opened → click)

REVENUE TRACKING:

- Tag every affiliate URL with UTM:
?utm_source=feelzlike&utm_medium=stay-card&utm_campaign={town}&utm_content={provider}
- Once affiliate dashboards arrive, you can attribute commissions back to the click

PRIVACY:

- Cookie banner already exists (per the audit – "We use cookies for essential features..."). Keep it; ensure analytics only fires after consent.
- Privacy policy page: list what's tracked, retention, opt-out

Run pnpm typecheck. Smoke-test by triggering an error and confirming Sentry receives it.

What to commit: feat(observability): Sentry coverage + product analytics

PART 9

Affiliate program guides

Eight programs analysed: how to apply, terms, recommended priority, and the prompt that activates IDs in code.

Part 9 — Affiliate programs (the revenue layer)

The single biggest opportunity in your app: the Stay section. Every property has booking links. Every booking link is an affiliate opportunity. Wired correctly, FeelZlike earns whether the user books today, tomorrow, or next month.

This part covers each of the 8 major programs with the realities you need to know — including some uncomfortable truths that affect your integration order. Then a single Replit prompt that flips all your placeholder IDs to live IDs in one config file.

The big picture (read this first)

After deep research, here's the integration order I recommend — **not** the order you might guess:

Tier	Program	Why this position
1	Integrate first	Trip.com Best 30-day cookie + deepest Japan inventory + competitive AU pricing
1	Integrate first	Agoda Highest transparent commission (4–7% direct on booking value); strong Japan
1	Integrate first	Booking.com Ubiquitous brand + inventory; despite session-only cookie, conversion is high
2	After MVP launch	Expedia Group Adds Hotels.com via single integration; 7-day cookie helps NSW conversions
2	After MVP launch	Rakuten Travel (via Involve Asia) Essential for authentic Nagano ryokan inventory
3	Lower priority	Tripadvisor Click-out model; weak for hotels but good for "Things to Do" links
3	Lower priority	Jalan 1% commission is too low to prioritise unless serving JP-language users
3	Speculative only	Airbnb Program is effectively closed (invite-only); apply but don't plan around it

Three realities most app founders learn the hard way

1. Booking.com has SESSION-ONLY tracking, not a 30-day cookie. If a user clicks your link, closes the tab, comes back tomorrow to book — **you earn nothing**. Their promotional materials are ambiguous about this; affiliates discover it after months of "missing" commissions. Plan accordingly.

2. Airbnb's Associates program closed in March 2021. The replacement "Creators" program is invite-only via Impact.com. Apply speculatively, but do not include Airbnb revenue in any forecast.

3. Rakuten Travel's English-language site lost affiliate commissions on 31 May 2022. Only the Japanese-language site (`travel.rakuten.co.jp`) earns. The clean workaround: apply via **Involve Asia** (English-language APAC affiliate network) — 1–3 day approval, no Japanese entity, no Japanese bank account required.

1. Booking.com Affiliate Partner Program

Apply: [Booking.com Affiliate Program V2](#) (Australian applicants are routed through [Awin](#))

Commission: 25–40% of Booking.com's margin (tiered by completed stays/month). Real-world effective rate: ~3–6% of booking value at lower volumes.

Monthly stayed bookings	Your share of margin
0–50	25%
51–150	30%
151–500	35%
501+	40%

Cookie: Session-only (the critical gotcha). Last-click attribution.

Payment: €100 minimum, monthly with 2-month lag. EUR/USD/GBP only — AUD not a payout currency. Direct bank transfer or PayPal.

Approval: Working website required, AU-friendly via Awin, 1 day–2 weeks. W-8BEN tax form for AU entities.

Deep-link format:

```
https://www.booking.com/searchresults.html?ss=Yudanaka&aid=YOUR_AID&label=feelzlike-yudanaka-stay
```

For a specific property:

```
https://www.booking.com/hotel/jp/property-name.html?aid=YOUR_AID
```

Always set the `label` **parameter** — it's your tracking string. Use it to identify which town/page generated the click (e.g., `feelzlike-jindabyne-stay-card-lakecrackenback`).

For Japan deep-linking: Add `language=ja` and `currency=JPY` .

Watch out for: Session-only cookie + 2-month payment lag + €100 threshold means revenue takes time. Cancellations don't pay. Booking.com's frequent A/B tests can disrupt your conversion flow without notice.

2. Agoda Partners

Apply: [Agoda Partners direct signup](#) — or via [Involve Asia](#) for APAC applicants.

Commission: 4–7% of pre-tax booking value (tiered, transparent — better than Booking.com's margin model at lower volumes).

Monthly completed bookings	Commission
1–50	4%
51–100	4.5%
101–200	5%
201–1,000	6%
1,000+	7%

Cookie: 24 hours (very short — second-shortest after Booking.com's session-only).

Payment: \$200 USD minimum (highest threshold of any program). Bank transfer only — no PayPal as of 2025. Pay-at-Agoda bookings paid 1 month after checkout; Pay-at-Hotel bookings paid 2 months after.

Approval: Working site/app, 1–3 days standard, 15 business days for API access. No JP entity required; AU-friendly.

Deep-link format:

```
https://www.agoda.com/partners/partnersearch.aspx?cid=YOUR_CID&hid=HOTEL_ID&checkin=2026-08-01&checkout=2026-08-03&currency=JPY
```

For Japan: Add `lang=ja-jp¤cy=JPY` . Agoda's Japan inventory and pricing is excellent for Nagano.

Watch out for: \$200 USD threshold + 24hr cookie + tier resets monthly = restart at 4% every month unless you scale fast.

3. Trip.com Partners (the dark horse)

Apply: [Trip.com Partners](#) — if "Access Denied", email affiliation@trip.com.

Commission: Up to 7% on hotels (tiered), up to 7% on car rentals, up to 5% on tours/activities. Base rate ~4–5% for new affiliates.

Cookie: 30 days — the best of any major hotel program.

Payment: No publicly stated minimum; PayPal and bank transfer; monthly with ~45-day validation lag.

Approval: A few days. "Everyone can join" per their PH FAQ — low barrier. Working site required. AUD supported for Australian users.

Deep-link format:

```
https://www.trip.com/hotels/list?city=Hakuba&locale=en-XX&curr=JPY&Allianceid=YOUR_ID&Sid=YOUR_SID
```

For Japan: This is the strongest program for Japan-market monetisation alongside Rakuten. Trip.com (Ctrip-owned) has invested heavily in Japan inventory — coverage of Nagano ryokan, hotels, and resort lodges is excellent.

Why Trip.com is your priority pick:

- Best cookie window of all programs (30 days)
- Up to 7% commission
- Strong Japan + AU inventory
- Low approval barrier

Watch out for: Lower brand recognition in AU than Booking.com; portal access can be flaky (use email fallback); slower affiliate support; PPC restrictions (no bidding on Trip.com brand keywords).

4. Expedia Group Affiliate Program (Expedia + Hotels.com)

Apply: [Expedia Group Affiliates](#) — single program covers Expedia.com, Hotels.com, Vrbo.

Commission: Flat ~4% on hotels (no volume tiering for standard affiliates). 6% on cruises, 4% on activities, 0% on flights.

Cookie: 7 days (real persistent cookie — middle of the pack but better than Booking.com or Agoda).

Payment: ~\$50 USD minimum, monthly, **~90-day holding period** post-stay (longest payout lag of any program). Bank transfer / PayPal — confirm at signup.

Approval: Working site, 1–2 weeks. AU-friendly, W-8BEN required.

Deep-link format:

```
https://www.expedia.com/Hotel-Search?destination=Jindabyne&affiliateId=YOUR_ID
```

For Hotels.com:

```
https://www.hotels.com/search.do?q-destination=Jindabyne&affiliateId=YOUR_ID
```

Why include it: Single integration unlocks Hotels.com inventory (separate brand recognition for some users). 7-day cookie is helpful. Strong AU presence.

Watch out for: Hotels.com's "10 nights free" loyalty reward was discontinued in 2023, which has reportedly hurt conversion. Japan inventory thinner than Agoda/Booking.com for ryokan.

5. Rakuten Travel Affiliate (via Involve Asia)

Apply (recommended pathway): [Involve Asia](#) — sign up as affiliate, then apply for the Rakuten Travel campaign.

Apply (direct, harder): [Rakuten Travel English program page](#) — but this requires a Rakuten Japan account, primarily Japanese-language interface.

Commission: 5% of booking value (paid in cash via Involve Asia; in Rakuten Points if you go direct).

Cookie: 30 days. Last-click. Commission confirmed one day after guest checks out.

Payment via Involve Asia: ~\$10–\$50 USD threshold (varies by method), PayPal / bank transfer / eWallet.

Approval via Involve Asia: 1–3 days, English-language, no JP entity or JP bank account required.

Deep-link format (via Involve Asia):

```
https://t.involve.asia/t/[PROGRAM_ID]?url=https://travel.rakuten.co.jp/hotel/[HOTEL_ID]/
```

Why this matters for FeelZlike: Rakuten Travel has the deepest inventory of authentic Japanese ryokan, minshuku, and small ski lodges in Nagano — accommodation that simply isn't on Booking.com or Agoda. The Yudanaka, Shibu Onsen, and Yomase listings in your dataset are heavily covered on Rakuten.

Watch out for: Rakuten Travel's English-language site lost affiliate support 31 May 2022 — **only** [travel.rakuten.co.jp](#) **(Japanese)** earns commissions. Australian users will land on a Japanese-language interface, which may hurt conversion. Best suited for Japanese-speaking users or travellers comfortable with Google Translate flows.

6. Tripadvisor Affiliate Program

Apply: [Tripadvisor Affiliates](#) — requires a [CJ Affiliate](#) account first.

Commission model is unusual:

- **Hotels:** 50% of Tripadvisor's revenue **per click-out** (not per booking) — averages **\$0.15–\$0.75 per click-out**. CPC-hybrid, not CPA.
- **Experiences (Things To Do):** 3% of booking value when booked directly on Tripadvisor.
- **Restaurants:** **0% — no affiliate commission on restaurant reservations.**

Cookie: 14 days, last-click.

Payment: \$50 USD minimum via CJ, monthly ~30 days after billing period.

Approval: Need a CJ account first (live website with custom domain required for CJ). Then apply to Tripadvisor campaign within CJ. A few days to 2 weeks.

Deep-link format:

```
https://www.tripadvisor.com/Hotel_Review-gXXXXXX-dYYYYYY-Reviews-Hotel_Name-City.html?m=YOUR_TRACKING_CODE
```

Why include it: Good for the "Things To Do" / Explore section once that's built — Snow Monkey Park tickets, Yamanouchi onsen tours, Snowy Mountains backcountry guides, etc.

Watch out for: Despite being a major restaurant discovery platform, **Tripadvisor pays nothing on restaurant clicks** — so don't wire it into the Eat section expecting revenue. Hotel revenue depends on click volume, not bookings — disconnects you from actual conversion quality.

7. Jalan.net (lowest priority)

Apply (via ASP — required):

- **A8.net** — most beginner-friendly Japanese ASP, English signup, no JP entity required
- **ValueCommerce** — stricter approval, English available

Commission: 1% of booking value on accommodation. 3.3% on activities. ¥500–700 per golf booking. **The lowest of all 8 programs.**

Cookie: Approximately 30 days (not officially documented in English).

Payment: A8.net minimum ¥1,000 (~\$7 USD), bank transfer or PayPal for international publishers.

Approval: A8.net 1–3 days platform + a few days for the Jalan campaign. ValueCommerce stricter — requires privacy policy/ToS on your site, 3–7 days.

Deep-link format (via A8.net):

```
https://px.a8.net/svt/ejp?a8mat=YOUR_A8_CODE&a8ejppredirect=https://www.jalan.net/yad[HOTEL_ID]/
```

Why deprioritise: 1% commission is mathematically poor for an English-language app — on a ¥20,000/night booking (~\$130 AUD) you earn ¥200 (~\$1.30). Plus the booking interface is entirely Japanese — bad UX for Australian users heading to Nagano.

When to integrate: If/when you build a Japanese-language UX for domestic Japanese users, Jalan becomes worthwhile because of the deep ryokan inventory not found elsewhere.

8. Airbnb (apply speculatively, don't plan around it)

Apply: **Airbnb Associates** — but be aware:

The Associates program closed permanently 31 March 2021. The replacement Creators program runs via Impact.com and is invite-only. Commission rates are not publicly disclosed; visible only post-approval.

Historical context (for reference): ~25–30% of Airbnb's service fee = approximately 3–5% of booking value. 28-day cookie. Monthly payments.

Current reality: Most applicants are rejected or get no response. The program targets large media publishers and creators.

What to do: Apply once. Move on. Build your revenue forecast around the other 7 programs. If accepted later, that's upside.

Comparison table

Program	Effective hotel commission	Cookie	Payment threshold	Approval	Best for
--- --- --- --- --- ---	Trip.com	up to 7% (tiered)	30 days	None stated	Easy Priority #1 — Japan + AU, best cookie
	Agoda	4–7% pre-tax (tiered)	1 day	\$200 USD	Easy Priority #2 — Japan, transparent rates
	Booking.com	~3–6% of margin	Session only	€100 / 2mo lag	Easy via Awin Priority #3 — brand + inventory
	Expedia Group	~4% flat	7 days	~\$50 USD	Easy NSW; adds Hotels.com
	Rakuten Travel	5% of value	30 days	~\$10–50 via Involve	Easy via Involve Asia
	Tripadvisor	\$0.15–0.75 per click-out; 3% Experiences	14 days	\$50 USD	Easy via CJ Things To Do, NOT restaurants
	Jalan	1%	~30 days	¥1,000	Easy via A8.net JP-language users only
	Airbnb	Undisclosed (historic 3–5%)	28d historic	Unknown	Very Hard Speculative

Application checklist (do this in order)

Week 1: Apply to Tier 1 (lowest effort, highest impact):

- ☐ **Trip.com Partners** (or email affiliation@trip.com if portal denies access)
- ☐ **Agoda Partners** direct
- ☐ **Booking.com via Awin** (Australian-routed)

Week 2: Apply to Tier 2:

- ☐ **Expedia Group Affiliates**
- ☐ **Involve Asia** (then apply to Rakuten Travel campaign within it)

Week 3: Apply to Tier 3 (optional):

- ☐ **CJ Affiliate** then **Tripadvisor** within CJ
- ☐ **A8.net** then **Jalan** campaign within it (only if pursuing JP-language UX)
- ☐ **Airbnb Associates / Creators** (set expectations: probably no response)

While you wait for approvals (1–2 weeks each):

- Wire the affiliate helper code with placeholder IDs (Sprint 2 in this playbook does exactly this)
- Build out the rest of the app
- As each approval lands, swap one ID in the config file (next prompt)

Compliance & disclosures (do not skip)

Australia: ACCC requires disclosure of affiliate relationships. Add a footer disclosure link: "Some links on FeelZlike are affiliate links — if you book through them we may earn a commission. This never affects what you pay."

Japan: As of October 2023, Japan's anti-stealth-marketing law requires clear "PR" or "PR (koukoku)" labelling on affiliate-linked content shown to Japanese users. The Stay cards must show a small "PR" tag near affiliate buttons when the user's locale is JP.

EU users (if any): GDPR requires consent before tracking cookies set by affiliate clicks. Your existing cookie banner needs to gate affiliate-tracking cookies behind explicit consent.

Prompt 9.1 — Activate affiliate IDs [ASSISTANT]

GOAL

When the first affiliate approvals arrive, you flip the IDs from `null` to live values in one config file. This prompt walks you through doing it correctly, including the per-region label tagging that lets you track which towns/cards generate revenue.

Open `artifacts/feelzlike/src/lib/affiliateLinks.ts` and paste:

```
I've been approved for one or more of these affiliate programs and need to activate the IDs.
```

```
Please:
```

1. Update the `AFFILIATE_IDS` object – set each approved provider's value from `null` to the actual ID I'll provide:
 - `booking_com`: `'aid=NUMBER'` (the AID from Awin/Booking.com dashboard)
 - `agoda`: `'cid=NUMBER'` (the CID from Agoda Partners)
 - `airbnb`: not yet – keep `null` unless approved
 - `expedia`: `'affiliateId=...'` (from Expedia Group dashboard)
 - `hotels_com`: `'affiliateId=...'` (same Expedia Group ID – Hotels.com uses the same)
 - `trip_com`: `'Allianceid=...&Sid=...'` (BOTH parameters from Trip.com)
 - `rakuten_travel`: from Involve Asia – they provide a tracked redirect URL pattern; update the rakuten builder to wrap the destination URL with the Involve Asia redirect:
`https://t.involve.asia/t/{PROGRAM_ID}?url={ENCODED_DESTINATION}`
 - `jalan`: from A8.net – update the jalan builder to wrap with A8 redirect:
`https://px.a8.net/svt/ejp?a8mat={A8_MAT}&a8ejpreirect={ENCODED_DESTINATION}`
 - `tripadvisor`: from CJ – update to use CJ click tracking wrapper:
`https://www.tkqlhce.com/click-{ACCOUNT_ID}-{PROGRAM_ID}?url={ENCODED_DESTINATION}`
2. Move every affiliate ID into Replit Secrets (NOT hardcoded):
 - `VITE_BOOKING_AFFILIATE_ID`
 - `VITE_AGODA_AFFILIATE_ID`
 - `VITE_EXPEDIA_AFFILIATE_ID`
 - `VITE_TRIP_AFFILIATE_ID`
 - `VITE_TRIP_SID`
 - `VITE_RAKUTEN_INVOLVE_PROGRAM_ID`
 - `VITE_JALAN_A8_MAT`
 - `VITE_TRIPADVISOR_CJ_ACCOUNT`
 - `VITE_TRIPADVISOR_CJ_PROGRAM`
 - The `affiliateLinks.ts` module reads from `import.meta.env.VITE_*` with `null` fallback.
3. Add the per-link tracking label parameter to every booking URL:
 - Format: `feelzlike-{town}-{provider}-{stay_id}`
 - Example: `feelzlike-yudanaka-booking-yorozuya`
 - This lets you analyse in each affiliate dashboard which town and which property converts best.
4. Add a "PR" badge to StayCard affiliate buttons when the user's locale is JP (Japan stealth-marketing law compliance, October 2023). The badge is a small pill labelled "PR" next to the booking button row.

```
5. Add a global affiliate disclosure to the site footer: "Some booking links on FeelZlike are affiliate links – if you book through them we may earn a commission. This never affects the price you pay." Link to a /legal/affiliate-disclosure page with full disclosure copy.

6. Update affiliateStatus() so the Settings/About page can show which programs are active – useful for the operator (you).

7. Run pnpm typecheck. Test: open a stay card, inspect the Booking.com link in DevTools – should now have aid=YOUR_ID and label=feelzlike-{town}-booking-{stayId}.

Do not commit secrets. Add VITE_*_AFFILIATE_* to .env.example with empty placeholders so other developers know what to set.
```

What to commit: feat(affiliate): activate live IDs + tracking labels + disclosures

How to test:

- 1 Open browser DevTools on a stay card
- 2 Right-click "Book on Trip.com" → Copy link → paste in a text editor
- 3 Verify the URL contains your Allianceid , Sid , AND the feelzlike-{town}-trip-{stayId} label
- 4 Repeat for each provider you've activated
- 5 Make a test booking through each one (a refundable booking somewhere) — confirm it shows up in the affiliate dashboard within 24–48 hours

If it goes wrong:

- "Click registers in dashboard but no commission on booking" → most likely a cookie issue. Booking.com is session-only — the user must complete in-session. Other providers: check whether ad-blockers are stripping the affiliate parameters. Test in incognito with no extensions.
- "Trip.com link works but no Sid in URL" → both Allianceid AND Sid are required for proper tracking. Check both env vars are set.
- "Involve Asia link redirects to login page" → the program ID is wrong, or the campaign isn't approved yet on the Involve Asia side.

PARTS 10 & 11

Appendices

Schema reference, URL cheatsheet, workflow tips, glossary.

Part 11 — Appendices

Appendix A — Full data schema reference

Common fields (all stays + eats)

| Field | Type | Notes | |---|---|---| | `id` | string (slug) | URL-safe unique identifier | | `name` | string | Display name | | `name_local` | string \| null | Kanji/katakana for Japan, null for NSW | | `type` | enum | Stay: ryokan/hotel/lodge/apartment/airbnb/hostel · Eat: izakaya/ramen/cafe/restaurant/bar/bakery/pub/grocery/fuel | | `town` | TownSlug | jindabyne / berridale / cooma / yudanaka / shibu_onsen / yomase | | `region` | RegionSlug | snowy_mountains / yamanouchi | | `country` | enum | AU / JP | | `short_description` | string | One-line tagline | | `long_description` | string | 2-3 sentences | | `address` | string | Full street address | | `lat` / `lng` | number | Coordinates | | `phone` | string \| null | International format preferred | | `website` | string \| null | Official URL | | `price_band` | enum | \$ / \$\$ / \$\$\$ / \$\$\$\$ | | `photos` | string[] | Source URLs (verified) | | `source_urls` | string[] | Where data was sourced from |

Stay-specific fields

| Field | Type | Notes | |---|---|---| | `room_count` | number \| null | Approximate | | `amenities` | string[] | Free-form tags | | `english_spoken` | enum | yes / limited / no | | `check_in` / `check_out` | string \| null | HH:mm format | | `booking_links` | object | Per-provider URLs (built by affiliateLinks helper) |

Japan stay extras

| Field | Type | Notes | |---|---|---| | `onsen` | enum | none / private / public / both | | `tattoo_policy` | enum | allowed / private_only / not_allowed / unknown | | `meal_plan` | enum | none / breakfast / dinner / kaiseki / half_board / full_board | | `yukata_provided` | bool | | | `walk_min_to_yudanaka_station` | number \| null |

NSW stay extras

| Field | Type | Notes | |---|---|---| | `drying_room` | bool \| null | | | `ski_storage` | bool \| null | | | `pet_friendly` | bool \| null | | | `distance_to_skitube_km` | number \| null | | | `distance_to_thredbo_km` | number \| null | | | `distance_to_perisher_km` | number \| null | | | `self_contained` | bool \| null | Has kitchen |

Eat-specific fields

| Field | Type | Notes | |---|---|---| | `cuisine` | string[] | E.g., ['ramen', 'shinshu'] | | `hours` | object | Keyed by day name | | `last_order_time` | string \| null | HH:mm — critical for ski-day planning | | `reservation` | enum | required / recommended / not_needed / not_accepted | | `reservation_link` | string \| null | Tabelog/Hotpepper/OpenTable/direct URL | | `payment` | enum | cash_only / cards_accepted / both | | `english_menu` | enum | yes / picture_menu / no | | `signature_dishes` | string[] | Free-form |

Japan eat extras

| Field | Type | Notes | |---|---|---| | `vegetarian_friendly` | enum | yes / limited / no | | | `kid_friendly` | bool \| null |

NSW eat extras

Field	Type	Notes
apres_ski	bool	Open after lift close, has alcohol
takeaway	bool	
groceries	bool	For IGA/supermarket entries

Computed fields (added at build time)

Field	Type	Notes
nearest_mountain	MountainSlug	Computed from lat/ing
drive_min_to_nearest_mountain	number	
drive_min_to_each_mountain	object	All mountains in region

Appendix B — Affiliate URL pattern cheat sheet

Quick reference for hand-crafting or debugging affiliate links.

```

Booking.com search
https://www.booking.com/searchresults.html?ss={QUERY}&aid={AID}&label={LABEL}

Booking.com property
https://www.booking.com/hotel/{country}/{slug}.html?aid={AID}&label={LABEL}

Agoda search
https://www.agoda.com/partners/partnersearch.aspx?cid={CID}&hid={HOTEL_ID}&checkin={DATE}&checkout={DATE}&currency={CCY}

Trip.com search
https://www.trip.com/hotels/list?city={CITY}&locale={LOCALE}&curr={CCY}&Allianceid={ID}&Sid={SID}

Expedia search
https://www.expedia.com/Hotel-Search?destination={QUERY}&affiliateId={ID}

Hotels.com search
https://www.hotels.com/search.do?q-destination={QUERY}&affiliateId={ID}

Rakuten Travel via Involve Asia
https://t.involve.asia/t/{PROGRAM_ID}?url={ENCODED_https://travel.rakuten.co.jp/hotel/HOTEL_ID/}

Jalan via A8.net
https://px.a8.net/svt/ejp?a8mat={A8_MAT}&a8ejppredict={ENCODED_https://www.jalan.net/yadHOTEL_ID/}

Tripadvisor via CJ
https://www.tkqlhce.com/click-{ACCOUNT_ID}-{PROGRAM_ID}?url={ENCODED_TRIPADVISOR_URL}

```

Always URL-encode the destination URL when wrapping in a redirect tracker.

Always add a `label / subId / Sid` style tracking string when the provider supports it — it lets you slice analytics by town and property in their dashboard.

Appendix C — Recommended Replit workflow

Agent vs Assistant — when to use which

| Task type | Use | |---|---| | Build a new feature touching 3+ files | **Agent** | | Fix a bug you understand | **Assistant** (open the file, ask) | | Refactor a single component | **Assistant** | | Add a new page + route + component + hook | **Agent** | | Update affiliate IDs in one file | **Assistant** | | "Read the codebase and propose a fix for X" | **Agent** |

Checkpoint discipline

Before every prompt:

```
git add -A
git commit -m "checkpoint: before prompt {X.Y} {description}"
```

BASH

If a prompt produces something you don't want:

```
git reset --hard HEAD
```

BASH

Periodically push to GitHub (or a remote backup):

```
git remote add origin <your-repo-url> # one-time
git push origin main
```

BASH

Branches for risky prompts

For Sprint 4+ prompts that touch core logic, work on a branch:

```
git checkout -b sprint-4-personalisation
# ... run prompts ...
git checkout main
git merge sprint-4-personalisation # only if it works
```

BASH

Reading the diff

When Agent proposes changes:

- 1 Click each modified file
- 2 Read the diff — don't skim
- 3 Watch for: hardcoded URLs (should be config), removed error handling, removed types
- 4 Reject anything you don't understand — ask Agent to explain or simplify

When prompts go wrong

Every prompt in this playbook has an "If it goes wrong" section. Use it. The recovery patterns are:

- 1 Run the suggested fix
- 2 If still broken: `git reset --hard HEAD`, simplify the prompt, try again
- 3 If fundamentally stuck: open the file in question, paste the error into Assistant, work it through manually

Appendix D — Glossary

| Term | Meaning | |---|---| | **AID / CID / Sid** | Affiliate ID parameters (Booking / Agoda / Trip.com respectively) | | **API spec** | Your `lib/api-spec` OpenAPI definition — source of truth for endpoints | | **Codegen** | Auto-generating Zod schemas + React Query hooks from the API spec | | **CPA / CPC** | Cost per acquisition / cost per click — affiliate payment models | | **EPC** | Earnings per click — affiliate performance metric | | **GOLD/SILVER/BRONZE Powder Window** | Quality grades for upcoming ski windows in the hourly forecast | | **Half-board (Japan)** | Ryokan rate including breakfast + kaiseki dinner | | **Involve Asia** | English-language APAC affiliate network — your route to Rakuten Travel | | **JST / AEDT** | Japan Standard Time / Australian Eastern Daylight Time | | **Kaiseki** | Multi-course traditional Japanese meal served at ryokan | | **Kuyu meguri** | Shibu Onsen's tradition of bathing at all 9 public bathhouses | | **PWA** | Progressive Web App — installable to home screen, offline-capable | | **Region isolation** | The principle that NSW data must never appear on JP pages and vice versa | | **Ryokan** | Traditional Japanese inn, typically with onsen and meals included | | **Session-only cookie** | Affiliate tracking that resets when the user closes their browser (Booking.com) | | **Skitube** | Underground railway from Bullocks Flat to Perisher Valley | | **Today's Call** | FeelZlike's live ranked comparison of mountains in a region | | **VAPID keys** | Voluntary Application Server Identification — required for web push | | **Wind-hold** | When a ski lift is closed due to high wind |

Appendix E — Where to ask for help

- **Replit issues:** [Replit Community](#)
- **Booking.com affiliate support:** Through Awin's partner manager (assigned after approval)
- **Agoda affiliate support:** partners@agoda.com
- **Trip.com:** affiliation@trip.com
- **Involve Asia:** support@involve.asia (responsive within 1 business day)
- **Open-Meteo (free weather API):** open-meteo.com — generous limits, no key needed
- **Sentry:** Their docs are excellent for React + Vite setup

Closing note

This playbook is a starting point, not a contract. As you build, you'll discover details that demand changes — features that are easier or harder than expected, user feedback that reorders priorities, affiliate programs that approve quickly or never. Treat the prompts as well-formed first drafts; refine them based on what you learn.

The most important thing: **ship the bug fixes first** (Sprint 1). Everything else can wait. Trust takes weeks to build and one bad screen to lose.

Build well.

You've reached the end.

Now open Replit, paste Prompt 0.1, and start the climb.

[FeelZlike](#) · What does the mountain feel like today?